

TypeScript: JavaScript With Types

Notes and exercises for all 14 chapters

Made on October 5, 2026

Contents

About this course

Level 1: The Basics

- 01 Getting Started
- 02 Basic Types
- 03 Arrays, Tuples and Objects
- 04 Type Aliases and Interfaces
- 05 Functions
- 06 Unions and Narrowing
- 07 Classes

Level 2: Reusable Types

- 08 Generics
- 09 Utility Types
- 10 keyof, typeof and Mapped Types

Level 3: Real Projects

- 11 tsconfig and Modules
- 12 TypeScript in the Browser
- 13 Typing Async Code and APIs
- 14 Project: Typed Shopping Cart

Hints

About this course

TypeScript is JavaScript plus **types**: labels that say what kind of value each thing should hold. A tool reads those labels and warns you about mistakes *before* you run your code.

This folder only covers what TypeScript **adds**. Everything else (variables, loops, functions, classes, async...) works exactly like in the JavaScript course, so finish that first, or at least Level 1 and 2.

The hints for the exercises are in the [Hints section](#) at the end. Try each exercise on your own first, and read one hint at a time.

Level 1: The Basics

Goal: add types to everyday JavaScript and understand what TypeScript is telling you.

01 Getting Started

What is it?

TypeScript is JavaScript with **types** added. A **type** is a label that says what kind of value something should hold: a number, some text, a list of products.

A tool called the **type checker** reads those labels and points out mistakes *before* you run the code.

Why does it matter?

In JavaScript, many bugs only show up while the program runs, sometimes in front of a user:

```
const price = "5"; // oops, text instead of a number
console.log(price * 2 + 1); // 11 (works by luck)
console.log(price + 1);    // "51" (wrong, and no error!)
```

You met this exact bug in JavaScript chapter 38. TypeScript catches it while you're typing. Your editor underlines the mistake in red, like a spell-checker.

That's why most professional JavaScript projects now use TypeScript: fewer bugs, better autocomplete in VS Code, and code that explains itself.

Real-world example

Think of the labels on moving boxes, from JavaScript chapter 02:

Moving house	TypeScript
A box labeled "Books"	<code>let title: string</code>
A friend checks the label before packing	The type checker
"Hey, you're putting a lamp in the Books box!"	A type error
The labels don't change what's inside	Types are removed before the code runs

The checker is a helpful friend, not a guard. It warns you, but it can't stop you. That last row matters, and you'll see why below.

How it works

Set up your playground (once)

You'll write all your TypeScript practice in one folder. Create a folder called `playground` inside this `TypeScript` folder, open a terminal in it, and run:

```
npm init -y
npm install --save-dev typescript
```

Then two small edits:

1. In `package.json`, change `"type": "commonjs"` to `"type": "module"` (same as JavaScript chapter 29).
2. Create a file called `tsconfig.json` with these settings:

```
{
  "compilerOptions": {
    "target": "esnext",
    "module": "nodenext",
    "lib": ["esnext", "dom"],
    "strict": true,
    "noEmit": true,
    "allowImportingTsExtensions": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true
  }
}
```

You don't need to understand every line yet. [Chapter 11](#) explains them. The important one is `"strict": true`, which turns on all the useful checks.

Your first TypeScript file

TypeScript files end in `.ts` instead of `.js`. Create `playground/hello.ts`:

```
const price: number = 5;
console.log("Price:", price * 2); // prints: Price: 10
```

The new part is `: number`. It's a **type annotation**: a label saying "price must always be a number". Everything else is plain JavaScript.

There are two separate steps: **run** the code, and **check** the types.

```
node hello.ts
npx tsc
```

- `node hello.ts` runs the file. Node 24 understands `.ts` files directly: it removes the types and runs the JavaScript that's left.
- `npx tsc` runs the **TypeScript compiler**, `tsc`. With our settings, it doesn't create any files. It only checks every `.ts` file in the folder and reports problems. No output means no problems.

Your first type error

Now break the rule on purpose:

```
let age: number = 30;
age = "thirty";
// ✖ Type 'string' is not assignable to type 'number'.
console.log(age);
```

Run `npx tsc` and you'll see:

```
hello.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
```

Read it like a JavaScript error message: file `hello.ts`, line 2, column 1. "Not assignable" means "you can't put this value in that box". In VS Code, the same message appears when you hover over the red squiggly line.

Tip: in this course, `// ✖` in a code example means "TypeScript reports this error here".

Types disappear when the code runs

Here's the surprise. Run the broken file with `node hello.ts`:

```
thirty
```

It runs! Node removes the types *without checking them*. Types only exist while you write code. They never exist at run time.

So the rule is: **always run `npx tsc` too**, or keep an eye on the red squiggles in VS Code. A program with type errors can still run, but it's a program with a known bug.

You don't have to label everything

TypeScript is good at working types out by itself. This is called **inference**:

```
let score = 10; // TypeScript infers: score is a number
score = "ten";
// ✗ Type 'string' is not assignable to type 'number'.
```

No annotation, same protection. You'll learn when to write types and when to let TypeScript infer them in [chapter 02](#).

Common mistakes

1. Thinking `node` checks your types

`node file.ts` runs code even when it has type errors. Checking is `npx tsc`'s job (or VS Code's squiggles).

2. Running `npx tsc hello.ts` with a file name

```
error TS5112: tsconfig.json is present but will not be loaded if files are specified on
commandline. Use '--ignoreConfig' to skip this error.
```

When you give `tsc` a file name, it ignores your `tsconfig.json`. Run plain `npx tsc` instead: it checks every file, with your settings.

3. Forgetting `"type": "module"`

With `"type": "commonjs"`, your `import` lines won't work with Node, just like in JavaScript chapter 29.

4. Running `tsc` in the wrong folder

`tsc` looks for `tsconfig.json` in the folder your terminal is in. Open the terminal in `playground`.

Quick recap

- TypeScript is JavaScript plus types. A type says what kind of value something should hold.
 - `: number` after a name is a type annotation. TypeScript can also infer types by itself.
 - `node file.ts` runs your code. `npx tsc` checks the types. They're separate steps.
 - Types disappear at run time, so a file with type errors still runs. Always check with `tsc`.
-

Next: try the [exercises](#), then move on to [02 Basic Types](#).

Exercises

How to do these:

- Do these in your `playground` folder. Make a subfolder for each chapter, like `playground/ch01/`, and one file per exercise (`ex1.ts`, `ex2.ts`, ...).
 - Run a file with `node ch01/ex1.ts`, and check all your types with `npx tsc` (from inside `playground`).
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Set up and say hello

Set up your playground exactly as the notes describe. Then create `ch01/ex1.ts` that prints:

```
Hello, TypeScript!
```

Run it with `node`, then run `npx tsc` and confirm it prints nothing (no errors).

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Label the boxes

Write a program for a cinema ticket with three variables, each with a type annotation: the movie title (text), the seat number (a number), and whether it's a 3D showing (true/false).

Print them so the output is:

```
Movie: Inside Out 2
Seat: 14
3D: false
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Make TypeScript complain

Copy your Exercise 2 file to `ex3.ts`. Then, **on purpose**, cause three different type errors, one for each variable, by assigning a value of the wrong type. (Change `const` to `let` so you can reassign.)

1. Run `npx tsc` and read each error. Write down the line number and the message for each one.
2. Now run `node ch01/ex3.ts`. Does it run? Write one sentence explaining why.
3. Fix the errors so `npx tsc` is quiet again.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): Catch the chapter 38 bug

In JavaScript, this cart total is wrong, because the quantity came from a form as text:

```
const unitPrice = 5;
const quantity = "3"; // came from an input box
const extraItems = 2;
console.log("Items:", quantity + extraItems); // prints: Items: 32
```

Rewrite it in TypeScript so that:

1. The type checker would have caught the bug, if the quantity were labeled as a number.
2. You convert the text to a number properly (remember JavaScript chapter 05).
3. The final program prints:

```
Items: 5
Total: 25
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

02 Basic Types

What is it?

The **basic types** are TypeScript's names for JavaScript's everyday values: `string`, `number`, `boolean`, `null` and `undefined`. Plus two special ones: `any` (turn checking off) and `unknown` (check before you use it).

Why does it matter?

Once TypeScript knows a value's type, it can tell you what you're allowed to do with it. Text has `.toUpperCase()`, numbers don't. So instead of finding out at run time with a `TypeError`, you find out while typing.

You also get **autocomplete**: type a dot after a string in VS Code, and it lists every string method. No more guessing method names.

Real-world example

Think of the plugs and sockets in your house:

Plugs and sockets	TypeScript
Each plug has a shape	Each value has a type
A socket accepts one shape	A variable or parameter accepts one type
A UK plug won't go in a US socket	<code>"thirty"</code> won't go in a <code>number</code> box
A universal adapter fits anything, but gives no protection	<code>any</code>

How it works

Annotations for the basic types

You met `: number` in chapter 01. The other basic types work the same way:

```
const city: string = "Kathmandu";
const temperature: number = 23.5; // whole numbers and decimals are both "number"
const isRaining: boolean = false;

console.log(city, temperature, isRaining); // prints: Kathmandu 23.5 false
```

They're the same types you learned with `typeof` in JavaScript chapter 03, always written in lowercase.

Inference: let TypeScript do the work

Most of the time, you don't need to write the type. TypeScript sees the starting value and works it out:

```
let city = "Kathmandu"; // TypeScript infers: string
let temperature = 23.5; // number

city = 42;
// ✗ Type 'number' is not assignable to type 'string'.
```

Hover over `city` in VS Code, and it shows `let city: string`. The protection is exactly the same as with an annotation.

So when should you write types yourself? A simple rule:

- **Let TypeScript infer** when you give a variable its value straight away.
- **Write the type** when TypeScript can't see the value yet: function parameters ([chapter 05](#)), empty arrays, and variables you fill in later.

```
let winner: string; // no value yet, so say what it will hold
winner = "Maya";
console.log(winner); // prints: Maya
```

The checker knows what each type can do

This is where TypeScript really pays off:

```
let price = 25;
console.log(price.toUpperCase());
// ✗ Property 'toUpperCase' does not exist on type 'number'.
```

In plain JavaScript, that's a `TypeError` at run time. In TypeScript, it's a red squiggle before you even save.

It also catches typos in method names:

```
let name = "sam";
console.log(name.toUppercase());
// ✗ Property 'toUppercase' does not exist on type 'string'. Did you mean 'toUpperCase'?
```

null and undefined

With `"strict": true`, a `string` can't secretly be `null` or `undefined`:

```
let nickname: string = "Sam";
nickname = null;
// ✗ Type 'null' is not assignable to type 'string'.
```

If something really can be missing, you say so with `|` ("or"):

```
let nickname: string | null = null; // text, or nothing yet
nickname = "Sam";
console.log(nickname); // prints: Sam
```

That `|` is called a **union type**. It's so useful that [chapter 06](#) is all about it.

any : switching the checker off

`any` means "this can be anything, don't check it":

```
let data: any = "hello";
data = 42;
data.fly(); // no error from TypeScript... but it crashes when it runs
```

`any` is the universal adapter: it fits everything and protects nothing. You'll see it in older code and quick experiments. Avoid writing it yourself.

unknown : the safe version of any

Sometimes you really don't know the type yet, like data from a user or an API. Use `unknown`. It accepts anything, but won't let you *use* the value until you check what it is:

```
let input: unknown = "hello";

console.log(input.toUpperCase());
// ✗ 'input' is of type 'unknown'.

if (typeof input === "string") {
  console.log(input.toUpperCase()); // fine: inside this if, input is a string
}
```

That `typeof` check is called **narrowing**: TypeScript follows your `if` and knows the type inside it. More in [chapter 06](#).

Type	Accepts any value?	Can you use it freely?
<code>any</code>	Yes	Yes, with no protection
<code>unknown</code>	Yes	No, check it first

Common mistakes

1. Using capital letters

```
const title: String = "Dune"; // works, but it's the wrong type
```

`String`, `Number` and `Boolean` (capitalized) are JavaScript's wrapper objects, not the basic types. Always use lowercase: `string`, `number`, `boolean`.

2. Annotating everything

```
const count: number = 5; // the annotation adds nothing here
```

TypeScript already knows `5` is a number. Extra annotations are just noise. Save them for places inference can't see.

3. Reaching for `any` to make an error go away

It makes the red line disappear, but it also makes the bug disappear from view, not from your program. Use `unknown` and check the value, or fix the real type.

4. Forgetting that some values can be `null`

If a value can be missing, include `| null` in its type. Then TypeScript reminds you to handle the empty case.

Quick recap

- The basic types are `string`, `number`, `boolean`, `null` and `undefined`, all lowercase.
- TypeScript infers types from starting values. Write types when it can't see the value.
- The checker knows what each type can do, so typos and wrong methods get caught early.
- In strict mode, missing values must be allowed on purpose, like `string | null`.
- `any` turns checking off. Prefer `unknown`, which makes you check before you use it.

Next: try the [exercises](#), then move on to [03 Arrays, Tuples and Objects](#).

Exercises

How to do these:

- Work in `playground/ch02/`, one file per exercise (`ex1.ts`, `ex2.ts`, ...).
- Run with `node ch02/ex1.ts`, and check types with `npx tsc` from inside `playground`.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Infer or annotate?

For each line, decide: does it need a type annotation, or can TypeScript infer it? Write the file with annotations **only** where they're needed, and make `npx tsc` quiet.

```
// no-check
let username = "sam_99";
let loginCount;           // filled in later
let isAdmin = false;
let lastLogin;            // filled in later, text like "2026-09-24"

loginCount = 3;
lastLogin = "2026-09-24";
console.log(username, loginCount, isAdmin, lastLogin);
```

Expected output:

```
sam_99 3 false 2026-09-24
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Fix the red lines

This weather report has three type errors. Find them with `npx tsc`, and fix the **values**, not the types:

```
// no-check
const city: string = "Pokhara";
let temperature: number = "24";
let isSunny: boolean = "yes";
let humidity: number = 70;
humidity = null;

console.log(city, temperature, isSunny, humidity);
```

Expected output after fixing (use `0` for the humidity):

```
Pokhara 24 true 0
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A value that can be missing

A library app stores who borrowed a book. When nobody has it, the value is `null`.

1. Declare `let borrowedBy` so it can hold a name **or** `null`, starting as `null`.
2. Print the message below with a check: if it's `null`, print `Available`, otherwise print `Borrowed by <name>`.
3. Set it to `"Priya"` and print the message again.

Expected output:

```
Available
Borrowed by Priya
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): unknown input

Pretend this value came from a user, so you can't trust its type:

```
// no-check
const input: unknown = " hello world ";
```

Write code that:

- If `input` is a string, prints it trimmed and in capital letters.
- If `input` is a number, prints it doubled.
- Otherwise, prints `Unsupported input` .

Expected output (for the string above):

```
HELLO WORLD
```

Change `input` to `21` , then to `true` , and check you get `42` and `Unsupported input` .

Rule: don't use `any` .

Stuck? The hint is in the [Hints section](#) at the end.

03 Arrays, Tuples and Objects

What is it?

This chapter shows how to describe **groups** of values:

- **Array types** describe a list where every item has the same type, like `string[]`.
- **Tuple types** describe a short list with a fixed length, where each position has its own type, like `[string, number]`.
- **Object types** describe which properties an object has, and the type of each one.

Why does it matter?

Most real data is lists and objects: a cart full of products, a user profile, a playlist. Typing them means TypeScript can catch a missing property, a misspelled one, or the wrong kind of item sneaking into a list.

Real-world example

Everyday thing	TypeScript
An egg carton: every slot holds an egg	An array: <code>string[]</code>
A map pin: exactly one latitude and one longitude, in that order	A tuple: <code>[number, number]</code>
A passport: fixed fields, like name and date of birth	An object type: <code>{ name: string; born: string }</code>

How it works

Arrays

Write the item type, then `[]`:

```
const playlist: string[] = ["Blinding Lights", "Levitating"];
playlist.push("Flowers");
playlist.push(42);
// ❌ Argument of type 'number' is not assignable to parameter of type 'string'.
```

`string[]` reads as "an array of strings". You'll also see `Array<string>`, which means exactly the same thing ([chapter 08](#) explains that style).

With a starting value, TypeScript infers the array type:

```
const scores = [90, 75, 81]; // inferred: number[]
const total = scores.reduce((sum, score) => sum + score, 0);
console.log(total); // prints: 246
```

Notice that `sum` and `score` got types too. TypeScript knows they're numbers because `scores` is a `number[]`, so array methods from JavaScript chapter 13 are fully checked.

An **empty** array is the one case where you should write the type, because there's nothing to infer from:

```
const cart: string[] = [];
cart.push("apples");
console.log(cart); // prints: [ 'apples' ]
```

Arrays that shouldn't change

Add `readonly` to stop a list from being changed by accident:

```
const weekdays: readonly string[] = ["Mon", "Tue", "Wed", "Thu", "Fri"];
weekdays.push("Sat");
// ✗ Property 'push' does not exist on type 'readonly string[]'.
```

It's like the "locked glass case" of `const` from JavaScript chapter 02, but for the items inside, too.

Tuples

A **tuple** is an array with a fixed length, where each position has a meaning and a type:

```
const location: [number, number] = [27.7172, 85.324]; // Latitude, Longitude
const entry: [string, number] = ["Asha", 92]; // name, score

console.log(entry[0], "scored", entry[1]); // prints: Asha scored 92
```

TypeScript checks the length and each position:

```
const point: [number, number] = [1, 2, 3];
// ✗ Type '[number, number, number]' is not assignable to type '[number, number]'.
```

Tuples work nicely with array destructuring (JavaScript chapter 15):

```
const entry: [string, number] = ["Asha", 92];
const [player, score] = entry; // player: string, score: number
console.log(`${player}: ${score}`); // prints: Asha: 92
```

Use tuples for small, fixed pairs. For anything with more than two or three parts, an object with named properties is easier to read.

Object types

Describe an object by listing its properties and their types, separated by `;`:

```
const book: { title: string; pages: number } = {
  title: "Dune",
  pages: 412,
};
console.log(book.title); // prints: Dune
```

A missing property is caught:

```
const book: { title: string; pages: number } = { title: "Dune" };
// ✗ Property 'pages' is missing in type '{ title: string; }' but required in type '{ title: string; pages: number; }'.
```

And so is an extra one, which is often a typo:

```
const book: { title: string; pages: number } = {
  title: "Dune",
  pages: 412,
  author: "Frank Herbert",
  // ✗ Object Literal may only specify known properties, and 'author' does not exist in type '{ title: string; pages: number; }'.
};
```

Writing the same object type again and again gets long. In [chapter 04](#), you'll give it a name and write it once.

Optional properties

Add `?` after a property name when it doesn't have to be there:

```
const book: { title: string; subtitle?: string } = { title: "Dune" };
console.log(book.subtitle); // prints: undefined
```

`subtitle?: string` means "a string, or not there at all". TypeScript treats its type as `string | undefined`, so it reminds you to check before using it:

```
const book: { title: string; subtitle?: string } = { title: "Dune" };
console.log(book.subtitle.toUpperCase());
// ❌ 'book.subtitle' is possibly 'undefined'.
console.log(book.subtitle?.toUpperCase()); // fine: optional chaining from JavaScript chapter 11
```

Read-only properties

`readonly` stops a property from being changed after the object is created. Great for IDs:

```
const user: { readonly id: number; name: string } = { id: 7, name: "Sam" };
user.name = "Samira"; // fine
user.id = 8;
// ❌ Cannot assign to 'id' because it is a read-only property.
```

Arrays of objects

The most common shape in real apps: a list of records.

```
const products: { name: string; price: number }[] = [
  { name: "Mug", price: 12 },
  { name: "Poster", price: 8 },
];

const names = products.map((product) => product.name); // string[]
console.log(names); // prints: [ 'Mug', 'Poster' ]
```

Common mistakes

1. Forgetting the type on an empty array

```
// no-check
const cart = []; // TypeScript can't tell what will go in here
```

Always type empty arrays: `const cart: string[] = [];`

2. Trusting array indexes too much

```
const colors: string[] = ["red", "green"];
const third = colors[2]; // TypeScript says: string
console.log(third);      // prints: undefined
```

TypeScript assumes an index gives you a real item, even when you read past the end. Check the length, or use `.at()` and handle `undefined`. (The setting `noUncheckedIndexedAccess` makes TypeScript stricter here. [Chapter 11](#) covers settings.)

3. Using commas and semicolons the wrong way round

In a **type**, properties are usually separated by `;`. In an **object value**, they're separated by `,`. (TypeScript accepts commas in types too, but `;` is the common style.)

4. Using a tuple when an object would be clearer

`["Asha", 92, true, "2026-09-24"]` makes readers guess what each position means. `{ name, score, passed, date }` says it.

Quick recap

- `string[]` is an array of strings. Type empty arrays yourself; otherwise, let TypeScript infer.
- `readonly string[]` can't be changed.
- A tuple like `[number, number]` has a fixed length and a type for each position.
- `{ title: string; pages: number }` describes an object. Missing and extra properties are errors.
- `?` makes a property optional, and `readonly` stops it from changing.

Next: try the [exercises](#), then move on to [04 Type Aliases and Interfaces](#).

Exercises

How to do these:

- Work in `playground/ch03/`, one file per exercise.
- Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): A typed shopping list

Create an empty, typed shopping list. Add `"bread"`, `"milk"` and `"eggs"` with `push`, then print how many items there are and the list joined with commas.

Expected output:

```
3 items: bread, milk, eggs
```

Then try pushing a number, read the error, and remove that line again.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Map pins

Store three places as tuples of `[name, latitude, longitude]`, in an array of tuples:

- Kathmandu, 27.7172, 85.324
- Pokhara, 28.2096, 83.9856
- Lumbini, 27.4833, 83.2767

Loop over them with destructuring and print:

```
Kathmandu is at 27.7172, 85.324
Pokhara is at 28.2096, 83.9856
Lumbini is at 27.4833, 83.2767
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A typed movie list

Create an array of movies. Each movie has a `title` (string), a `year` (number), and an optional `rating` (number).

```
// no-check
{ title: "Inside Out 2", year: 2024, rating: 8 }
{ title: "Dune: Part Two", year: 2024 }
{ title: "Coco", year: 2017, rating: 9 }
```

Print each movie, with `not rated` when there's no rating:

Inside Out 2 (2024): 8/10
Dune: Part Two (2024): not rated
Coco (2017): 9/10

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Challenge): Bank accounts with read-only IDs

Create an array of bank accounts. Each account has a `readonly id` (number), an `owner` (string) and a `balance` (number).

1. Start with two accounts: id 1, owner "Asha", balance 500; id 2, owner "Ben", balance 120.
2. Write a loop that adds 5% interest to every balance (round to 2 decimal places with `Math.round(x * 100) / 100`).
3. Print every account.
4. Try to change an account's `id` inside the loop, and read the error. Then remove that line.

Expected output:

```
#1 Asha: 525  
#2 Ben: 126
```

Stuck? The hint is in the [Hints section](#) at the end.

04 Type Aliases and Interfaces

What is it?

A **type alias** gives a name to a type, with the `type` keyword. An **interface** gives a name to an object's shape, with the `interface` keyword.

Both let you write a type once and reuse it everywhere.

Why does it matter?

In chapter 03, you wrote `{ title: string; pages: number }` over and over. That's tiring, and if a book gets a new property, you'd have to update every copy.

A named type fixes both problems. You write `Book` once, use it everywhere, and change it in one place. The name also documents your code: `function lend(book: Book)` says a lot more than a long inline shape.

Real-world example

Think of a **form template**, like a blank job application:

Paper forms	TypeScript
The blank template, with labeled fields	<code>interface Applicant { name: string; ... }</code>
A filled-in copy	An object, like <code>{ name: "Asha", ... }</code>
Checking a copy against the template	The type checker
A longer form that adds fields to the basic one	<code>interface Manager extends Employee</code>

How it works

Type aliases

`type` + a name + `=` + any type:

```

type Book = {
  title: string;
  pages: number;
};

const dune: Book = { title: "Dune", pages: 412 };
const hobbit: Book = { title: "The Hobbit", pages: 310 };

console.log(dune.pages + hobbit.pages); // prints: 722

```

Type names use **PascalCase** (every word starts with a capital), like `Book` or `ShoppingCart`, so they stand out from variables.

An alias can name *any* type, not only objects:

```

type Coordinates = [number, number]; // a tuple
type Price = number; // a clearer name for a number

const kathmandu: Coordinates = [27.7172, 85.324];
const ticket: Price = 12;
console.log(kathmandu, ticket); // prints: [ 27.7172, 85.324 ] 12

```

Interfaces

An interface does the same job, but only for object shapes. Notice there's no `=`:

```

interface Product {
  name: string;
  price: number;
  inStock?: boolean; // optional, like in chapter 03
}

const mug: Product = { name: "Mug", price: 12 };
const poster: Product = { name: "Poster" };
// ✖ Property 'price' is missing in type '{ name: string; }' but required in type 'Product'.

```

The error message now uses the name `Product`. Much easier to read than a long inline shape.

Extending: building on another type

Often one type is "another type, plus a few things". An interface can **extend** another one:

```

interface Employee {
  name: string;
  salary: number;
}

interface Manager extends Employee {
  teamSize: number;
}

const boss: Manager = { name: "Priya", salary: 5000, teamSize: 8 };
console.log(`${boss.name} manages ${boss.teamSize} people`); // prints: Priya manages 8 people

```

A `Manager` has everything an `Employee` has, plus `teamSize`. It's like inheritance with `extends` in JavaScript chapter 27, but for types.

Type aliases do the same with `&`, called an **intersection** ("this AND that"):

```

type Employee = { name: string; salary: number };
type Manager = Employee & { teamSize: number };

const boss: Manager = { name: "Priya", salary: 5000, teamSize: 8 };
console.log(boss.salary); // prints: 5000

```

Named types in functions and arrays

This is where names really pay off:

```

interface Product {
  name: string;
  price: number;
}

const cart: Product[] = [
  { name: "Mug", price: 12 },
  { name: "Poster", price: 8 },
];

function describe(product: Product): string {
  return `${product.name} costs ${product.price}`;
}

console.log(describe(cart[0])); // prints: Mug costs $12

```

(Typing function parameters like `product: Product` is the topic of [chapter 05](#).)

type or interface ?

For object shapes, both work, and the differences are small:

	type	interface
Object shapes	Yes	Yes
Other types (unions, tuples, a name for <code>number</code>)	Yes	No
Combining	<code>&</code>	<code>extends</code>
Can be "reopened" to add more properties later	No	Yes

That last row is called **declaration merging**: if you write `interface Product` twice, TypeScript combines them. It's mostly used by libraries, and you'll rarely need it.

This course's rule: use `interface` for object shapes, and `type` for everything else. Many teams use `type` for everything, and that's fine too. Just be consistent within a project.

Common mistakes

1. Adding `=` to an interface, or leaving it off a type

```
// no-check
interface Book = { title: string } // wrong: interfaces have no =
type Book { title: string }        // wrong: types need =
```

Remember: `type Name = ...` but `interface Name { ... }`.

2. Starting type names with a lowercase letter

`interface product` works, but it looks like a variable. Use `Product`.

3. Copying the same shape into many places

If you see the same `{ ... }` object type twice, give it a name.

4. Thinking types exist when the code runs

```
// no-check
interface Book { title: string }
console.log(Book); // ❌ 'Book' only refers to a type, but is being used as a value here.
```

Types disappear before your code runs (chapter 01). You can't print them, loop over them, or pass them to functions.

Quick recap

- `type Name = ...` names any type. `interface Name { ... }` names an object shape.
 - Use PascalCase for type names: `Book`, `ShoppingCart`.
 - Interfaces combine with `extends`. Type aliases combine with `&`.
 - Named types make error messages and function signatures much easier to read.
 - This course uses `interface` for objects and `type` for everything else.
-

Next: try the [exercises](#), then move on to [05 Functions](#).

Exercises

How to do these:

- Work in `playground/ch04/`, one file per exercise.
 - Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Name that shape

This code repeats the same object type three times. Create one `interface Song` and use it everywhere, so the shape is written only once.

```
// no-check
const song1: { title: string; artist: string; seconds: number } = { title: "Flowers", artist:
  "Miley Cyrus", seconds: 200 };
const song2: { title: string; artist: string; seconds: number } = { title: "Levitating", artist:
  "Dua Lipa", seconds: 203 };
const playlist: { title: string; artist: string; seconds: number }[] = [song1, song2];

let total = 0;
for (const song of playlist) {
  total += song.seconds;
}
console.log(`Playlist length: ${total} seconds`);
```

Expected output:

```
Playlist length: 403 seconds
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Type aliases for clarity

A delivery app stores a location as a tuple and an ID as text. Create:

- `type Coordinates` for a `[latitude, longitude]` tuple.
- `type OrderId` for a string.
- `interface Delivery` with an `id` (`OrderId`), a `to` (`Coordinates`) and an optional `note` (string).

Create one delivery and print:

```
Order A-102 goes to 27.7172, 85.324
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Extending

A gym has members, and some members are personal trainers.

1. `interface Member` with `name` (string) and `visits` (number).
2. `interface Trainer` that extends `Member` and adds `clients` (an array of strings).
3. Create one trainer, and print:

```
Dana has 12 visits and trains 2 clients: Asha, Ben
```

Then try creating a `Trainer` without `clients`, and read the error. Remove that line again.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): The same thing, two ways

Rewrite Exercise 3 using only `type` aliases and `&` instead of interfaces and `extends`. The output must stay exactly the same.

Then, in a comment at the top of your file, answer in your own words:

1. Which version do you find easier to read, and why?
2. Name one thing a `type` can do that an `interface` can't.

Stuck? The hint is in the [Hints section](#) at the end.

05 Functions

What is it?

In TypeScript, you add types to a function's **inputs** (its parameters) and, optionally, its **output** (its return value). You can also describe a whole function as a type, like "a function that takes a number and returns a string".

Why does it matter?

Functions are where most mix-ups happen: calling with the arguments in the wrong order, forgetting one, or passing text instead of a number. Once a function's inputs are typed, every single call to it is checked, all over your project.

It also means you don't need to open a function to know how to use it. Hover over its name in VS Code, and the types tell you.

Real-world example

A vending machine:

Vending machine	Function types
"Coins only" slot	Parameter type: <code>coins: number</code>
A snack comes out	Return type: <code>: Snack</code>
The machine rejects a button, or a banknote	A type error at the call
"Exact change not needed" (optional)	Optional parameter: <code>change?: number</code>

How it works

Typing parameters

Parameters are the one place you should (almost) always write types, because TypeScript can't guess what callers will pass:

```
function tip(bill: number, percent: number) {  
    return bill * (percent / 100);  
}  
  
console.log(tip(50, 15)); // prints: 7.5
```

Now every call is checked:

```
function tip(bill: number, percent: number) {  
    return bill * (percent / 100);  
}  
  
tip(50);  
// ✖ Expected 2 arguments, but got 1.  
tip("50", 15);  
// ✖ Argument of type 'string' is not assignable to parameter of type 'number'.
```

If you leave a parameter's type out, strict mode complains:

```
function double(x) {  
    // ✖ Parameter 'x' implicitly has an 'any' type.  
    return x * 2;  
}
```

Return types

TypeScript infers the return type from your `return` statements. Hover over `tip` above, and it shows `function tip(bill: number, percent: number): number`.

You can also write it yourself, after the parentheses:

```
function formatPrice(price: number): string {  
    return `$$${price.toFixed(2)}`;   
}  
  
console.log(formatPrice(4.5)); // prints: $4.50
```

Writing the return type is a good habit for important functions: if you accidentally return the wrong thing, the error appears inside the function, where the mistake is.

```
function isAdult(age: number): boolean {
  return age >= 18 ? "yes" : "no";
  // ✗ Type 'string' is not assignable to type 'boolean'.
}
```

void : functions that return nothing

A function that only *does* something, like printing, returns nothing. Its return type is `void`:

```
function greet(name: string): void {
  console.log(`Hello, ${name}!`);
}

greet("Asha"); // prints: Hello, Asha!
```

Optional and default parameters

Add `?` for a parameter that callers can skip. Inside the function, it might be `undefined`:

```
function greet(name: string, title?: string): string {
  if (title) {
    return `Hello, ${title} ${name}`;
  }
  return `Hello, ${name}`;
}

console.log(greet("Sharma", "Dr.")); // prints: Hello, Dr. Sharma
console.log(greet("Sam"));           // prints: Hello, Sam
```

A default value (JavaScript chapter 09) makes a parameter optional too, and TypeScript infers its type from the default:

```
function tip(bill: number, percent = 15) { // percent: number
  return bill * (percent / 100);
}

console.log(tip(40));           // prints: 6
console.log(tip(40, 20));       // prints: 8
```

Optional parameters must come after the required ones.

Arrow functions

Same rules, same places:

```
const toCelsius = (fahrenheit: number): number => ((fahrenheit - 32) * 5) / 9;
console.log(toCelsius(212)); // prints: 100
```

Rest parameters

A rest parameter (JavaScript chapter 15) is an array, so its type is an array type:

```
function sum(...numbers: number[]): number {
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(sum(5, 10, 20)); // prints: 35
```

Function types

Sometimes you need to describe a *function itself*, like a callback. A **function type** looks like an arrow function, with types instead of code:

```
type PriceFormatter = (price: number) => string;

const asDollars: PriceFormatter = (price) => `$$${price.toFixed(2)}`;
const asRupees: PriceFormatter = (price) => `Rs. ${price}`;

function printPrices(prices: number[], format: PriceFormatter): void {
  for (const price of prices) {
    console.log(format(price));
  }
}

printPrices([3, 12.5], asDollars);
```

You'll see:

```
$3.00
$12.50
```

Notice that `(price) =>` needs no annotation: TypeScript already knows from `PriceFormatter` that `price` is a number. This is called **contextual typing**, and it's why callbacks for `map` and `filter` need no types either.

Common mistakes

1. Leaving parameters untyped

Parameter 'x' implicitly has an 'any' type. means "tell me what this is". Add a type.

2. Annotating callback parameters that are already known

```
const prices = [3, 12.5];
const doubled = prices.map((price: number) => price * 2); // the ": number" adds nothing
```

TypeScript already knows `price` is a number from the array. Let it infer.

3. Putting an optional parameter first

```
// no-check
function greet(title?: string, name: string) {}
// ❌ A required parameter cannot follow an optional parameter.
```

Required parameters first, optional ones last.

4. Mixing up `void` and `undefined` in your head

`void` means "don't use the return value". Use it for functions that only *do* something, like printing or saving.

Quick recap

- Always type parameters. TypeScript then checks every call: the number of arguments and their types.
- Return types are inferred, but writing them on important functions catches mistakes inside the function.
- `void` means the function returns nothing useful.
- `?` makes a parameter optional. A default value does too, and sets its type.
- `type Formatter = (price: number) => string` describes a function, which is handy for callbacks.

Next: try the [exercises](#), then move on to [06 Unions and Narrowing](#).

Exercises

How to do these:

- Work in `playground/ch05/`, one file per exercise.
- Run with `node`, and check types with `npm tsc`. Every exercise must pass `npm tsc` with no errors.

- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Add the types

This JavaScript works, but `npx tsc` complains about every parameter. Add parameter and return types.

```
// no-check
function areaOfRoom(width, length) {
  return width * length;
}

function describeRoom(name, width, length) {
  return `${name}: ${areaOfRoom(width, length)} m²`;
}

console.log(describeRoom("Kitchen", 3, 4));
```

Expected output:

```
Kitchen: 12 m²
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Optional and default

Write a function `makeTicket` for a cinema that takes:

- `movie` (required, text)
- `seat` (required, text)
- `price`, defaulting to `12`
- `snack` (optional, text)

It returns a line of text. Calling it like this:

```
// no-check
console.log(makeTicket("Coco", "F7"));
console.log(makeTicket("Coco", "F8", 9, "popcorn"));
```

should print:

```
Coco, seat F7, $12
Coco, seat F8, $9, with popcorn
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A function type for discounts

A shop has several discount rules. Each rule is a function that takes a price and returns the new price.

1. Create type `Discount = ...` for such a function.
2. Create three discounts: `noDiscount`, `tenPercentOff`, and `fiveOff` (takes \$5 off, but never goes below 0).
3. Write `applyDiscount(price: number, discount: Discount): number`.

```
// no-check
console.log(applyDiscount(40, noDiscount));
console.log(applyDiscount(40, tenPercentOff));
console.log(applyDiscount(3, fiveOff));
```

Expected output:

```
40
36
0
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): A typed grade report

Write these functions, with full types:

- `average(...scores: number[]): number` returns the average (and `0` if there are no scores).
- `letterGrade(score: number): string` returns "A" for 90+, "B" for 80+, "C" for 70+, otherwise "F".
- `report(name: string, ...scores: number[]): void` prints one line, using the other two functions.

```
// no-check
report("Asha", 92, 88, 95);
report("Ben", 70, 65);
report("Chen");
```

Expected output:

```
Asha: 91.67 (A)
Ben: 67.5 (F)
Chen: 0 (F)
```

Round the average to 2 decimal places for display, but don't show extra zeros (67.5 , not 67.50).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

06 Unions and Narrowing

What is it?

A **union type** says a value can be one of several types: `string | number` means "a string OR a number".

Narrowing is how TypeScript works out which one it is right now, by following your `if` checks.

A **literal type** is a type that allows only one exact value, like `"small"`. Unions of literals, like `"small" | "medium" | "large"`, are one of TypeScript's most useful tools.

Why does it matter?

Real data is often "this or that": an ID that's a number or a string, a result that's data or an error, an order that's "pending", "shipped" or "delivered".

Unions let you describe that exactly. Narrowing makes sure you handle every case before you use the value. And literal types turn typos like `"shiped"` into errors.

Real-world example

A coffee shop menu:

Coffee shop	TypeScript
"Pay by cash or card"	<code>type Payment = Cash Card</code>
The cashier checks which one you're holding before doing anything	Narrowing
Cup sizes: small, medium or large only	<code>type Size = "small" "medium" "large"</code>
"Can I get a huge?" "Sorry, that's not a size."	A type error

How it works

Union types

```
function printId(id: number | string): void {
  console.log(`Your ID is ${id}`);
}

printId(101);    // prints: Your ID is 101
printId("A-202"); // prints: Your ID is A-202
```

With a union, you can only use what **all** the members have in common:

```
function shout(id: number | string) {
  return id.toUpperCase();
  // ❌ Property 'toUpperCase' does not exist on type 'string | number'.
}
```

That's TypeScript protecting you: if `id` is a number, `toUpperCase` would crash.

Narrowing with `typeof`

Check the type first, and TypeScript follows along:

```
function formatId(id: number | string): string {
  if (typeof id === "string") {
    return id.toUpperCase(); // here, id is a string
  }
  return id.toFixed(0);      // here, id must be a number
}

console.log(formatId("a-202")); // prints: A-202
console.log(formatId(101));    // prints: 101
```

Hover over `id` on each line in VS Code, and you'll see its type change. After the `if` returns, TypeScript knows the string case is handled, so the rest is a number.

Other ways to narrow

Most normal JavaScript checks narrow types:

```
function welcome(name: string | null): string {
  if (name === null) {
    return "Welcome, guest!";
  }
  return `Welcome, ${name}!`; // name is a string here
}
```

```
console.log(welcome(null)); // prints: Welcome, guest!
console.log(welcome("Sam")); // prints: Welcome, Sam!
```

Check	Narrows...
<code>typeof x === "string"</code>	basic types
<code>x === null</code> , <code>if (x)</code>	away <code>null</code> and <code>undefined</code>
<code>Array.isArray(x)</code>	arrays
<code>"price" in x</code>	objects with a certain property
<code>x instanceof Date</code>	class instances

Literal types

A literal type allows exactly one value. On its own, that's not very useful. In a union, it's brilliant:

```
type Size = "small" | "medium" | "large";

function coffeePrice(size: Size): number {
  if (size === "small") return 3;
  if (size === "medium") return 4;
  return 5;
}

console.log(coffeePrice("medium")); // prints: 4
coffeePrice("huge");
// ✗ Argument of type '"huge"' is not assignable to parameter of type 'Size'.
```

VS Code also autocompletes the three sizes when you type `coffeePrice("`.

Remember from chapter 02 that `const` gives a literal type: `const size = "small"` has the type `"small"`, while `let size = "small"` has the type `string`.

Discriminated unions

Here's the pattern you'll use most. Several object types share one property, like `kind`, with a different literal value in each:

```
interface Cash {
  kind: "cash";
  amount: number;
}

interface Card {
  kind: "card";
  amount: number;
  last4: string;
}

type Payment = Cash | Card;

function describe(payment: Payment): string {
  switch (payment.kind) {
    case "cash":
      return `Cash: ${payment.amount}`;
    case "card":
      return `Card ending ${payment.last4}: ${payment.amount}`; // last4 exists here
  }
}

console.log(describe({ kind: "cash", amount: 20 }));
console.log(describe({ kind: "card", amount: 35, last4: "4242" }));
```

You'll see:

```
Cash: $20
Card ending 4242: $35
```

The shared `kind` property is called the **discriminant**: checking it tells TypeScript which type you have, so inside `case "card"`, it knows `last4` exists.

Never miss a case

What if someone adds a third payment type later? You can make TypeScript tell you about every `switch` that forgot it. The type `never` means "this can't happen":

```

interface Cash { kind: "cash"; amount: number }
interface Card { kind: "card"; amount: number }
interface Voucher { kind: "voucher"; code: string }
type Payment = Cash | Card | Voucher;

function describe(payment: Payment): string {
  switch (payment.kind) {
    case "cash":
      return "Cash";
    case "card":
      return "Card";
    default: {
      const unhandled: never = payment;
      // ✖ Type 'Voucher' is not assignable to type 'never'.
      return unhandled;
    }
  }
}

```

If every case is handled, nothing can reach `default`, and `payment` has the type `never` there. When a case is missing, the error names it: you forgot `Voucher`. Add `case "voucher"`, and the error goes away.

What about `enum`?

In other code, you'll see `enum`, an older way to make a set of named values:

```

// no-check
enum Size { Small, Medium, Large }

```

Node can't run `enum` directly (you'd get `TypeScript enum is not supported in strip-only mode`), and our `erasableSyntaxOnly` setting warns about it. A union of literals does the same job and is simpler, so this course uses those. If you need the values at run time too, use an object with `as const` ([chapter 10](#)).

Common mistakes

1. Using a member-only method without narrowing

Property `'x'` does not exist on type `'A' | 'B'` means "only one of them has this". Narrow first.

2. Typos in literal strings

```
type Status = "pending" | "shipped";
const status: Status = "shipped";
// ✗ Type '"shipped"' is not assignable to type 'Status'. Did you mean '"shipped"'?
```

This is exactly the kind of bug literal types catch, and TypeScript even suggests the fix. Plain `string` would have let it through.

3. Using `let` when you want a literal

`let size = "small"` is a `string`, so it won't fit a `Size` parameter later. Use `const`, or write `let size: Size = "small"`.

4. Forgetting the discriminant

If your object types don't share a `kind` (or `type`, or `status`) property with different literal values, TypeScript can't tell them apart in a `switch`. Add one.

Quick recap

- `A | B` means "A or B". You can only use what both have in common until you narrow.
- `typeof`, `=== null`, `in`, `Array.isArray` and `instanceof` checks all narrow the type.
- Unions of literals, like `"small" | "medium" | "large"`, allow only those exact values.
- A discriminated union shares a `kind` property, so a `switch` on `kind` narrows to the right type.
- A `never` check in `default` makes TypeScript tell you when a case is missing.

Next: try the [exercises](#), then move on to [07 Classes](#).

Exercises

How to do these:

- Work in `playground/ch06/`, one file per exercise.
 - Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Price tags

A price can arrive as a number (12) or as text ("12.5"). Write `toPrice(value: number | string): number` that always returns a number, then print the total of these prices:

```
// no-check
const prices = [12, "12.5", 3, "0.5"];
```

Expected output:

```
Total: 28
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Traffic light

Create type `Light = "red" | "yellow" | "green"` and a function `action(light: Light): string` that returns "Stop", "Slow down" or "Go".

```
// no-check
console.log(action("red"));
console.log(action("green"));
```

Expected output:

```
Stop
Go
```

Then try `action("blue")` and read the error. Remove that line again.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Order status messages

An online shop's orders are one of three kinds:

- pending (no extra data)
- shipped, with a `trackingCode`
- delivered, with a `deliveredOn` date string

Model them as a discriminated union on a `status` property, and write `message(order: Order): string`:

```
// no-check
console.log(message({ status: "pending" }));
console.log(message({ status: "shipped", trackingCode: "NP123" }));
console.log(message({ status: "delivered", deliveredOn: "2026-09-24" }));
```

Expected output:

```
We're packing your order.
On its way! Tracking code: NP123
Delivered on 2026-09-24.
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): Never forget a case

Take your Exercise 3 code and add the `never` check from the notes in a `default` branch.

1. Confirm `npx tsc` is still quiet.
2. Add a fourth kind, `cancelled`, with a `reason` string, to the `Order` union **without** updating the `switch`.
Read the error: which type does it name?
3. Handle the new case, so this line:

```
// no-check
console.log(message({ status: "cancelled", reason: "out of stock" }));
```

prints:

```
Cancelled: out of stock
```

Stuck? The hint is in the [Hints section](#) at the end.

07 Classes

What is it?

Classes in TypeScript are JavaScript classes (JavaScript chapter 27) with a few extras: typed properties, **access modifiers** (`public`, `private`, `protected`), `readonly`, and `implements`, which checks that a class follows an interface.

Why does it matter?

A class is a blueprint for many objects. If the blueprint has a mistake, like a property that's never set or a method with the wrong name, every object made from it has that mistake.

TypeScript checks the blueprint itself: every property is declared with a type, every property gets a starting value, and code outside the class can't touch what's marked private.

Real-world example

A bank account:

Bank	TypeScript class
Your account number never changes	<code>readonly accountNumber</code>
Only the bank can change your balance directly	<code>private balance</code>
Anyone can ask to deposit	<code>public deposit()</code>
Every branch must offer the same services	<code>implements BankServices</code>

How it works

Declare your properties

In TypeScript, a class lists its properties with their types, before the constructor:

```

class Pet {
  name: string;
  age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  describe(): string {
    return `${this.name} is ${this.age}`;
  }
}

const rex = new Pet("Rex", 3);
console.log(rex.describe()); // prints: Rex is 3

```

If you forget to give a declared property a value, strict mode tells you:

```

class Pet {
  name: string;
  // ✖ Property 'name' has no initializer and is not definitely assigned in the constructor.
}

```

A property can also get its value right where it's declared, like `visits = 0;`. Then TypeScript infers its type.

private and public

Everything is `public` by default: usable from anywhere. Mark a property `private` to allow it only inside the class:

```

class BankAccount {
  private balance = 0;

  deposit(amount: number): void {
    this.balance += amount;
  }

  getBalance(): number {
    return this.balance;
  }
}

const account = new BankAccount();
account.deposit(50);
console.log(account.getBalance()); // prints: 50
account.balance = 1000000;
// ❌ Property 'balance' is private and only accessible within class 'BankAccount'.

```

private or #private?

You learned JavaScript's own `#balance` in chapter 27. What's the difference?

	private balance	#balance
Checked by	TypeScript only	JavaScript itself
Still hidden when the code runs?	No, types disappear	Yes

TypeScript's `private` is a type check. Once the code runs, it's gone, like all types. `#balance` is real privacy, enforced by JavaScript. For new code, `#` is the safer choice. You'll see `private` a lot in existing code, so it's good to know both.

`protected` is a third option: like `private`, but child classes (made with `extends`) can use it too.

readonly

`readonly` properties can be set in the constructor, then never again:

```

class Ticket {
  readonly id: string;
  seat: string;

  constructor(id: string, seat: string) {
    this.id = id;
    this.seat = seat;
  }
}

const ticket = new Ticket("T-1", "F7");
ticket.seat = "F8"; // fine: seat can change
ticket.id = "T-2";
// ✗ Cannot assign to 'id' because it is a read-only property.

```

`implements` : promise to follow an interface

An interface can describe what a class must have. `implements` makes TypeScript check it:

```

interface Notifier {
  send(message: string): void;
}

class EmailNotifier implements Notifier {
  send(message: string): void {
    console.log(`Email: ${message}`);
  }
}

class SmsNotifier implements Notifier {
  send(message: string): void {
    console.log(`SMS: ${message}`);
  }
}

const notifiers: Notifier[] = [new EmailNotifier(), new SmsNotifier()];
for (const notifier of notifiers) {
  notifier.send("Your order has shipped");
}

```

You'll see:

```

Email: Your order has shipped
SMS: Your order has shipped

```

If a class forgets something, the error appears on the class itself:

```
interface Notifier {  
    send(message: string): void;  
}  
  
class PushNotifier implements Notifier {}  
// ✖ Class 'PushNotifier' incorrectly implements interface 'Notifier'.
```

This is the strategy pattern from JavaScript chapter 44, with TypeScript making sure every strategy has the same shape.

Abstract classes

An `abstract` class is a half-finished blueprint. You can't make objects from it directly. Child classes fill in the missing parts:

```
abstract class Shape {  
    abstract area(): number; // no body: child classes must write it  
  
    describe(): string {  
        return `Area: ${this.area().toFixed(1)}`;  
    }  
}  
  
class Circle extends Shape {  
    radius: number;  
  
    constructor(radius: number) {  
        super();  
        this.radius = radius;  
    }  
  
    area(): number {  
        return Math.PI * this.radius ** 2;  
    }  
}  
  
console.log(new Circle(2).describe()); // prints: Area: 12.6
```

`new Shape()` would give the error `Cannot create an instance of an abstract class.`

A shortcut you'll see elsewhere

In other TypeScript code, you'll often see this shortcut, called **parameter properties**:

```
// no-check
class Pet {
  constructor(public name: string, private age: number) {}
}
```

It declares and sets the properties in one line. But Node can't run it directly, and our `erasableSyntaxOnly` setting reports `This syntax is not allowed when 'erasableSyntaxOnly' is enabled.` So in this course, declare properties the long way. Just recognize the shortcut when you see it.

Common mistakes

1. Assigning a property you never declared

```
class Pet {
  constructor(name: string) {
    this.name = name;
    // ✗ Property 'name' does not exist on type 'Pet'.
  }
}
```

Declare it first: `name: string;` above the constructor.

2. Thinking `private` protects data at run time

`private` disappears when types are removed. For real protection, use `#`.

3. Using a class when a plain object would do

If your class has no methods and no private data, an `interface` and a plain object are simpler (the KISS principle).

Quick recap

- Declare class properties with types above the constructor, and give each one a starting value.
- `private` limits a property to inside the class, `protected` to the class and its children, and `public` is the default.
- TypeScript's `private` is only a type check. JavaScript's `#` is real privacy.
- `readonly` properties can only be set in the constructor.
- `implements` checks that a class matches an interface. `abstract` classes leave parts for child classes to fill in.

Next: try the [exercises](#), then move on to [08 Generics](#).

Exercises

How to do these:

- Work in `playground/ch07/`, one file per exercise.
- Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Type the class

Add property declarations and types so `npx tsc` is quiet:

```
// no-check
class Book {
  constructor(title, author, pages) {
    this.title = title;
    this.author = author;
    this.pages = pages;
  }

  summary() {
    return `${this.title} by ${this.author}, ${this.pages} pages`;
  }
}

console.log(new Book("Dune", "Frank Herbert", 412).summary());
```

Expected output:

```
Dune by Frank Herbert, 412 pages
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): A piggy bank

Write a `PiggyBank` class with:

- a readonly owner (string), set in the constructor,
- a private coins count, starting at 0,
- add(count: number): void, and total(): number.

```
// no-check
const bank = new PiggyBank("Maya");
bank.add(5);
bank.add(3);
console.log(`${bank.owner} has ${bank.total()} coins`);
```

Expected output:

```
Maya has 8 coins
```

Then try `bank.coins = 100;` and `bank.owner = "Sam";`, read both errors, and remove those lines.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Payment methods with implements

Create interface `PaymentMethod` with a method `pay(amount: number): string`. Then write two classes that implement it:

- `CashPayment` returns `Paid $<amount> in cash`.
- `CardPayment` takes the last 4 digits in its constructor and returns `Paid $<amount> with card ending <last4>`.

Put one of each in a `PaymentMethod[]` array, and pay 25 with each:

```
Paid $25 in cash
Paid $25 with card ending 4242
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): An abstract employee

Create an abstract class `Employee` with:

- a name (string), set in the constructor,

- an abstract method `monthlyPay(): number`,
- a normal method `payslip(): string` that returns `<name>: $<pay>`.

Then create two child classes:

- `SalariedEmployee`: yearly salary in the constructor, pays salary / 12.
- `HourlyEmployee`: hourly rate and hours in the constructor, pays rate × hours.

```
// no-check
const staff: Employee[] = [
  new SalariedEmployee("Priya", 60000),
  new HourlyEmployee("Ben", 20, 80),
];
for (const person of staff) {
  console.log(person.payslip());
}
```

Expected output:

```
Priya: $5000
Ben: $1600
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Level 2: Reusable Types

Goal: write flexible types once and reuse them everywhere.

08 Generics

What is it?

A **generic** is a type with a blank in it, which gets filled in later. You write the blank as `<T>` (short for "Type"), and TypeScript fills it in based on how you use it.

You've already used generics without knowing: `Array<string>` is the generic `Array` with its blank filled in as `string`.

Why does it matter?

Say you want a function that returns the first item of any list. Without generics, you have two bad choices:

- Write one version per type (`firstString`, `firstNumber`, `firstProduct` ...). Lots of copying.
- Use `any`, and lose all type checking.

Generics give you one function that works for every type *and* keeps the types. If you pass in a list of products, TypeScript knows you get a product back.

Real-world example

A **lunch box**:

Lunch box	Generics
The same box can carry a sandwich, rice, or fruit	One generic type: <code>Box<T></code>
Today it's labeled "sandwich"	<code>Box<Sandwich></code>

Lunch box	Generics
Open it, and you know what you'll find	TypeScript knows the type of what comes out
"Must fit in the fridge"	A constraint: <code>T extends ...</code>

How it works

A generic function

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}

const song = first(["Flowers", "Levitating"]); // song: string | undefined
const score = first([90, 75, 81]);             // score: number | undefined

console.log(song, score); // prints: Flowers 90
```

Read `first<T>(items: T[]): T` as: "for any type `T`, take an array of `T`, and give back a `T`".

You never said what `T` is. TypeScript **inferred** it from the argument: an array of strings means `T` is `string`. You can also fill it in yourself: `first<string>([...])`. That's rarely needed.

(`| undefined` is there because an empty array has no first item.)

Compare with `any`

```
function firstAny(items: any[]): any {
  return items[0];
}

const song = firstAny(["Flowers"]);
song.toFixed(2); // no error from TypeScript... but it crashes when it runs
```

With `any`, the type information is thrown away. With `T`, it flows through the function.

Generic interfaces and types

Types can have blanks too. A common one is an API response that can hold any kind of data:

```
interface ApiResponse<T> {
  ok: boolean;
  data: T;
}

interface User {
  name: string;
}

const userResponse: ApiResponse<User> = { ok: true, data: { name: "Asha" } };
const countResponse: ApiResponse<number> = { ok: true, data: 42 };

console.log(userResponse.data.name); // prints: Asha
console.log(countResponse.data + 1); // prints: 43
```

Write the interface once, reuse it for every kind of data. [Chapter 13](#) uses exactly this with `fetch`.

Generics you already know

You've written	It's the generic...
<code>string[]</code>	<code>Array<string></code>
An <code>async</code> function's result	<code>Promise<T></code> , like <code>Promise<number></code>
<code>new Map()</code> from JavaScript chapter 35	<code>Map<K, V></code> , like <code>Map<string, number></code>

```
const stock = new Map<string, number>(); // keys are strings, values are numbers
stock.set("mug", 12);
stock.set("poster", "lots");
// ❌ Argument of type 'string' is not assignable to parameter of type 'number'.
```

Constraints: "T, but it must have..."

Sometimes your function needs something from `T`. This fails, because not every type has `.length`:

```
function longest<T>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
  // ❌ Property 'length' does not exist on type 'T'.
}
```

Add a **constraint** with `extends`: "T can be any type, as long as it has a `length` number":

```
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}

console.log(longest("tea", "coffee")); // prints: coffee
console.log(longest([1, 2], [1, 2, 3])); // prints: [ 1, 2, 3 ]
longest(10, 20);
// ✗ Argument of type 'number' is not assignable to parameter of type '{ length: number; }'.
```

A very practical example: find any item by its `id`, whatever kind of item it is:

```
function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
  return items.find((item) => item.id === id);
}

const products = [
  { id: 1, name: "Mug", price: 12 },
  { id: 2, name: "Poster", price: 8 },
];

const found = findById(products, 2); // found: { id: number; name: string; price: number } |
  undefined
console.log(found?.name); // prints: Poster
```

More than one blank

Use a second letter for a second type:

```
function pair<A, B>(first: A, second: B): [A, B] {
  return [first, second];
}

const entry = pair("Asha", 92); // entry: [string, number]
console.log(entry); // prints: [ 'Asha', 92 ]
```

`T`, `U`, `K` (key) and `V` (value) are the usual letters. For clarity, you can use full names too, like `<Item>`.

Common mistakes

1. Using `any` when you mean "any type, but keep it"

If the output type depends on the input type, you want a generic, not `any`.

2. Adding generics that aren't needed

```
function greet<T extends string>(name: T): string {  
  return `Hi, ${name}`;  
}
```

`T` is only used once, so it adds nothing. `function greet(name: string)` is simpler. A generic is useful when it **connects** two things, like the input and the output.

3. Forgetting the constraint

Property `'x'` does not exist on type `'T'` means TypeScript doesn't know `T` has `x`. Add `T extends { x: ... }`.

Quick recap

- A generic is a type with a blank, written `<T>`, that gets filled in later.
- TypeScript usually infers `T` from the arguments, so the types flow through your function.
- Interfaces and types can be generic too: `ApiResponse<T>`, `Box<T>`.
- `Array<T>`, `Promise<T>` and `Map<K, V>` are generics you already use.
- `T extends { id: number }` is a constraint: any type, as long as it has what you need.

Next: try the [exercises](#), then move on to [09 Utility Types](#).

Exercises

How to do these:

- Work in `playground/ch08/`, one file per exercise.
- Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors, and must not use `any`.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): `last`

Write a generic `last<T>(items: T[]): T | undefined` that returns the last item.

```
// no-check
console.log(last(["Mon", "Tue", "Wed"]));
console.log(last([3, 1, 4]));
console.log(last([]));
```

Expected output:

```
Wed
4
undefined
```

Hover over the result of each call in VS Code, and check that the type matches.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): A typed box

Create `interface Box<T>` with a `label` (string) and `contents` (T). Make a `Box<string[]>` for books and a `Box<number>` for coins, then print:

```
Books box holds 2 items
Coins box holds 150
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Group by a key

Write `groupBy<T>(items: T[], getKey: (item: T) => string): Map<string, number>`, which counts how many items share each key.

```
// no-check
const orders = [
  { id: 1, city: "Kathmandu" },
  { id: 2, city: "Pokhara" },
  { id: 3, city: "Kathmandu" },
];
const counts = groupBy(orders, (order) => order.city);
console.log(counts);
```

Expected output:

```
Map(2) { 'Kathmandu' => 2, 'Pokhara' => 1 }
```

Notice that `order` in the callback needs no type: TypeScript knows it's one of the orders.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): A tiny typed store

Write a generic class `Store<T extends { id: number }>` that keeps a list of items and has:

- `add(item: T): void`
- `find(id: number): T | undefined`
- `remove(id: number): void`
- `count(): number`

```
// no-check
const store = new Store<{ id: number; title: string }>();
store.add({ id: 1, title: "Buy milk" });
store.add({ id: 2, title: "Call mum" });
store.remove(1);
console.log(store.count(), store.find(2)?.title, store.find(1));
```

Expected output:

```
1 Call mum undefined
```

Then try `store.add({ title: "No id" })` and read the error.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

09 Utility Types

What is it?

Utility types are ready-made generic types that come with TypeScript. Each one takes a type you already have and makes a new version of it: all properties optional, some properties removed, everything read-only, and so on.

Why does it matter?

Real apps need many *variations* of the same type. Take a `User` :

- The sign-up form needs every field.
- The "edit profile" form lets you change just some fields.
- The public profile must never include the password.

Without utility types, you'd copy `User` three times and keep all the copies in sync by hand. With them, you write `User` once and describe each variation in one line.

Real-world example

A **photo editing app**: you don't retake the photo to make a variation. You apply a filter to the original.

Photo filter	Utility type
Crop to just the faces	<code>Pick<User, "name" "email"></code>
Blur out one thing	<code>Omit<User, "password"></code>
"Make every part optional"	<code>Partial<User></code>
Lock the photo so it can't be edited	<code>Readonly<User></code>

The original stays the same. Each filter gives you a new version.

How it works

All the examples below use this type:

```
// no-check
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}
```

Partial<T> : every property optional

Perfect for updates, where you only send what changed:

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

function updateUser(user: User, changes: Partial<User>): User {
  return { ...user, ...changes }; // spread from JavaScript chapter 15
}

const sam: User = { id: 1, name: "Sam", email: "sam@example.com", password: "secret" };
const updated = updateUser(sam, { email: "sam@new.com" });
console.log(updated.email); // prints: sam@new.com
```

`Partial<User>` means `{ id?: number; name?: string; email?: string; password?: string }`, without you writing it.

Required<T> : the opposite

`Required<T>` makes every property required, even the ones marked `?`. Handy when a settings object has optional fields, but after you fill in the defaults, they're all there.

Pick<T, Keys> and Omit<T, Keys>

`Pick` keeps only the properties you name. `Omit` keeps everything *except* the ones you name. The property names are written as a union of literals ([chapter 06](#)):

```

interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

type PublicUser = Omit<User, "password">; // id, name, email
type LoginForm = Pick<User, "email" | "password">; // email, password

const profile: PublicUser = { id: 1, name: "Sam", email: "sam@example.com" };
const login: LoginForm = { email: "sam@example.com", password: "secret" };

const leaky: PublicUser = { id: 1, name: "Sam", email: "sam@example.com", password: "secret" };
// ✖ Object literal may only specify known properties, and 'password' does not exist in type
//   'PublicUser'.

```

That last line is TypeScript helping with security: a `PublicUser` can't accidentally carry a password (JavaScript chapter 51).

`Readonly<T>` : nothing can change

```

interface Settings {
  theme: string;
  fontSize: number;
}

const defaults: Readonly<Settings> = { theme: "light", fontSize: 16 };
defaults.fontSize = 20;
// ✖ Cannot assign to 'fontSize' because it is a read-only property.

```

Like `Object.freeze` from JavaScript chapter 43, but checked before the code runs. (It's shallow too: nested objects aren't locked.)

`Record<Keys, Value>` : a lookup table

`Record` describes an object where every key has the same type of value:

```

type Size = "small" | "medium" | "large";

const coffeePrices: Record<Size, number> = {
  small: 3,
  medium: 4,
  large: 5,
};

console.log(coffeePrices.medium); // prints: 4

```

With literal keys like `Size`, TypeScript also checks that none are missing:

```

type Size = "small" | "medium" | "large";

const teaPrices: Record<Size, number> = { small: 2, medium: 3 };
// ✖ Property 'large' is missing in type '{ small: number; medium: number; }' but required in
  type 'Record<Size, number>'.

```

With `string` keys, like `Record<string, number>`, it's a flexible dictionary (a `Map` from JavaScript chapter 35 is often better for those).

ReturnType<T> : the type a function returns

Sometimes a function already exists, and you want the type of what it returns without writing it again:

```

function createOrder(item: string, quantity: number) {
  return { item, quantity, createdAt: "2026-09-24" };
}

type Order = ReturnType<typeof createOrder>;
// Order is { item: string; quantity: number; createdAt: string }

const order: Order = createOrder("mug", 2);
console.log(order.quantity); // prints: 2

```

`typeof createOrder` means "the type of this function". You'll see more of this `typeof` in [chapter 10](#).

Quick reference

Utility	Gives you
<code>Partial<T></code>	All properties optional

Utility	Gives you
<code>Required<T></code>	All properties required
<code>Readonly<T></code>	All properties read-only
<code>Pick<T, "a" "b"></code>	Only <code>a</code> and <code>b</code>
<code>Omit<T, "a"></code>	Everything except <code>a</code>
<code>Record<K, V></code>	An object with keys <code>K</code> and values <code>V</code>
<code>ReturnType<typeof fn></code>	What <code>fn</code> returns
<code>NonNullable<T></code>	<code>T</code> without <code>null</code> and <code>undefined</code>

Common mistakes

1. Copying a type instead of deriving it

If `EditUserForm` is "User, but everything optional", write `Partial<User>`. When `User` changes, the variation updates by itself.

2. Misspelling a key in `Pick` or `Omit`

```
interface User {
  name: string;
  email: string;
}

type OnlyName = Pick<User, "nmae">;
// ✗ Type '"nmae"' does not satisfy the constraint 'keyof User'.
```

The error mentions `keyof`, which [chapter 10](#) explains.

3. Expecting `Readonly` to lock nested objects

`Readonly<T>` only locks the top level, like `Object.freeze`.

Quick recap

- Utility types make new versions of a type you already have, so you write the original once.
- `Partial` and `Required` flip whether properties are optional.
- `Pick` keeps some properties, and `Omit` removes some.

- `readonly` stops changes, and `Record` describes lookup tables.
- `ReturnType<typeof fn>` gives you the type a function returns.

Next: try the [exercises](#), then move on to [10 keyof, typeof and Mapped Types](#).

Exercises

How to do these:

- Work in `playground/ch09/`, one file per exercise.
- Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
- **Rule for all exercises:** write the base interface once, and build every variation with utility types. Don't copy properties by hand.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Edit a recipe

```
// no-check
interface Recipe {
  id: number;
  title: string;
  minutes: number;
  vegetarian: boolean;
}
```

Write `editRecipe(recipe: Recipe, changes: ...): Recipe` so that callers can change any fields they like. Then:

```
// no-check
const soup: Recipe = { id: 1, title: "Tomato soup", minutes: 30, vegetarian: true };
const faster = editRecipe(soup, { minutes: 20 });
console.log(faster.title, faster.minutes);
```

Expected output:

```
Tomato soup 20
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Hide the secret

```
// no-check
interface Account {
  id: number;
  username: string;
  email: string;
  passwordHash: string;
}
```

1. Create type `PublicAccount` without `passwordHash`.
2. Write `toPublic(account: Account): PublicAccount`.
3. Print the public version of one account.

Expected output (for username `maya_k`, id 3, email `maya@example.com`):

```
{ id: 3, username: 'maya_k', email: 'maya@example.com' }
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Opening hours

Create type `Day = "mon" | "tue" | "wed" | "thu" | "fri" | "sat" | "sun"`; and an `openingHours` object with a closing time for each day, typed with `Record`. Sunday is closed, stored as `null`.

Print:

```
mon: open until 18:00
sun: closed
```

Then delete one day from your object, and read the error.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): Settings with defaults

```
// no-check
interface Settings {
  theme: "light" | "dark";
  fontSize: number;
  language: string;
}
```

Write `loadSettings(saved: Partial<Settings>): Readonly<Settings>`. It fills in any missing setting from these defaults: `light`, `16`, `en`. The result must be read-only.

```
// no-check
const settings = loadSettings({ theme: "dark" });
console.log(settings);
```

Expected output:

```
{ theme: 'dark', fontSize: 16, language: 'en' }
```

Then try `settings.fontSize = 20;` and read the error.

Stuck? The hint is in the [Hints section](#) at the end.

10 keyof, typeof and Mapped Types

What is it?

These are tools for **building types out of other types and values**:

- `keyof` gives you the property names of a type, as a union.
- `typeof` (in a type position) gives you the type of a value.
- **Indexed access**, like `User["email"]`, gives you the type of one property.
- `as const` makes a value's type as exact as possible.
- **Mapped types** make a new type by going through every property of another one.

This is how utility types like `Partial` from chapter 09 are built.

Why does it matter?

The less you repeat yourself, the fewer things can get out of sync. These tools let you say "the keys of this type" or "the type of this object" instead of writing them out again. Change the original, and everything built from it updates by itself.

Real-world example

A **restaurant menu** and the order slips the waiters use:

Restaurant	TypeScript
The printed menu	An object value: <code>const menu = { soup: 5, pasta: 12 }</code>
The list of dishes you're allowed to order	<code>keyof typeof menu : "soup" "pasta"</code>
Add a dish to the menu, and the order slips update too	Types derived from values stay in sync

How it works

keyof : the property names

```
interface Product {  
  name: string;  
  price: number;  
  inStock: boolean;  
}  
  
type ProductKey = keyof Product; // "name" | "price" | "inStock"  
  
const sortBy: ProductKey = "price";  
const oops: ProductKey = "colour";  
// ✗ Type '"colour"' is not assignable to type 'keyof Product'.
```

Indexed access: the type of one property

Use square brackets on a *type* to get one property's type:

```
interface Product {  
  name: string;  
  price: number;  
}  
  
type Price = Product["price"]; // number  
  
const discount: Price = 5;  
console.log(discount); // prints: 5
```

Put them together: a safe property getter

`keyof` and generics ([chapter 08](#)) combine into something very useful:

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const mug = { name: "Mug", price: 12 };

const price = getProperty(mug, "price"); // price: number
const name = getProperty(mug, "name");   // name: string
console.log(name, price); // prints: Mug 12

getProperty(mug, "colour");
// ✗ Argument of type '"colour"' is not assignable to parameter of type '"name" | "price"'.
```

Read `K extends keyof T` as "K must be one of T's property names", and `T[K]` as "the type of that property".

typeof : the type of a value

In normal code, `typeof` is JavaScript's run-time check (JavaScript chapter 03). In a **type** position, it means "the type of this variable":

```
const defaultSettings = {
    theme: "light",
    fontSize: 16,
};

type Settings = typeof defaultSettings; // { theme: string; fontSize: number }

const mine: Settings = { theme: "dark", fontSize: 18 };
console.log(mine.theme); // prints: dark
```

Now the object is the single source of truth. Add a setting to `defaultSettings`, and `Settings` gets it too.

as const : the most exact type

Normally, TypeScript widens values to general types: `"light"` becomes `string`. `as const` keeps them exact and makes everything read-only:

```
const sizes = ["small", "medium", "large"] as const;
// sizes: readonly ["small", "medium", "large"]

type Size = (typeof sizes)[number]; // "small" | "medium" | "large"

const pick: Size = "medium";
console.log(sizes.length, pick); // prints: 3 medium
```

`(typeof sizes)[number]` means "the type of any item in this array". This trick gives you a list you can loop over at run time *and* a union type, from one line. It's the enum replacement promised in [chapter 06](#):

```
const Status = {
  Pending: "pending",
  Shipped: "shipped",
} as const;

type Status = (typeof Status)[keyof typeof Status]; // "pending" | "shipped"

function ship(status: Status): string {
  return status === Status.Pending ? "Shipping now" : "Already shipped";
}

console.log(ship(Status.Pending)); // prints: Shipping now
```

satisfies : check a value without changing its type

`satisfies` checks that a value matches a type, but keeps the value's exact inferred type:

```
type Size = "small" | "medium" | "large";

const prices = {
  small: 3,
  medium: 4,
  large: 5,
} satisfies Record<Size, number>;

console.log(prices.large); // prints: 5
```

If you forget a size or misspell one, TypeScript complains, like with a normal annotation.

Mapped types

A **mapped type** goes through every key of a type and builds a new property for each one. It looks like a loop inside a type:

```
interface Settings {
  theme: string;
  fontSize: number;
}

type Flags<T> = {
  [K in keyof T]: boolean; // for each key K in T, make a boolean property
};

type ChangedSettings = Flags<Settings>; // { theme: boolean; fontSize: boolean }

const changed: ChangedSettings = { theme: true, fontSize: false };
console.log(changed); // prints: { theme: true, fontSize: false }
```

Here's how `Partial` from chapter 09 is really built:

```
// no-check
type MyPartial<T> = {
  [K in keyof T]?: T[K]; // same keys, same types, but optional
};
```

You won't write mapped types every day. But when you see one in a library, you'll know it's "a loop over the keys".

Common mistakes

1. Mixing up the two `typeof` s

`typeof x === "string"` is JavaScript, and runs when the code runs. `type T = typeof x` is TypeScript, and only exists while type checking.

2. Forgetting `as const`

```
const sizes = ["small", "medium"];
type Size = (typeof sizes)[number]; // just string, not the exact sizes
```

Without `as const`, the array is a `string[]`, so the union collapses to `string`.

3. Over-engineering

These tools are powerful, which makes them tempting. If a simple `interface` does the job, use it (KISS). Reach for mapped types only when you'd otherwise repeat yourself a lot.

Quick recap

- `keyof T` is a union of `T`'s property names, and `T["key"]` is one property's type.
 - `K` extends `keyof T` with `T[K]` gives you type-safe property access.
 - `typeof value` in a type gives you the type of a value, so a value can be the source of truth.
 - `as const` keeps exact literal types. `(typeof arr)[number]` turns a list into a union.
 - `satisfies` checks a value against a type without changing its type.
 - Mapped types `([K in keyof T]: ...)` build new types by looping over keys.
-

Next: try the [exercises](#), then move on to [11 tsconfig and Modules](#).

Exercises

How to do these:

- Work in `playground/ch10/`, one file per exercise.
 - Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Menu keys

```
// no-check
const menu = { soup: 5, pasta: 12, salad: 7 };
```

1. Create `type Dish` as the union of the menu's keys, using `keyof` `typeof`.
2. Write `priceOf(dish: Dish): number`.
3. Print the price of pasta, then add `cake: 6` to the menu and check that `priceOf("cake")` works without changing `Dish`.

Expected output (after adding cake):

```
12
6
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): A list and a union from one line

A gym has membership levels `"basic"`, `"plus"` and `"premium"`.

1. Write them **once**, as an array with `as const`.
2. Create `type Level` from that array.
3. Loop over the array to print all levels, and write `isLevel(value: string): boolean`.

```
// no-check
console.log(isLevel("plus"), isLevel("gold"));
```

Expected output:

```
basic
plus
premium
true false
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A type-safe sort

Write `sortBy<T, K extends keyof T>(items: T[], key: K): T[]`, which returns a **new** sorted array (smallest first). Use it on:

```
// no-check
const books = [
  { title: "Dune", pages: 412 },
  { title: "Coco", pages: 96 },
  { title: "Emma", pages: 474 },
];
console.log(sortBy(books, "pages").map((book) => book.title));
console.log(sortBy(books, "title").map((book) => book.title));
```

Expected output:

```
[ 'Coco', 'Dune', 'Emma' ]  
[ 'Coco', 'Dune', 'Emma' ]
```

Then check that `sortBy(books, "author")` is an error.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): A form's error messages

```
// no-check  
interface SignUpForm {  
  username: string;  
  email: string;  
  age: number;  
}
```

1. Write a mapped type `FormErrors<T>` that has the same keys as `T`, each an **optional** `string` (the error message).
2. Write `validate(form: SignUpForm): FormErrors<SignUpForm>` that adds a message when the username is shorter than 3 characters, when the email has no `@`, and when the age is under 13.

```
// no-check  
console.log(validate({ username: "al", email: "al.example.com", age: 30 }));  
console.log(validate({ username: "alice", email: "alice@example.com", age: 30 }));
```

Expected output:

```
{ username: 'Too short', email: 'Missing @' }  
{}
```

Stuck? The hint is in the [Hints section](#) at the end.

Level 3: Real Projects

Goal: use TypeScript in real Node and browser projects.

11 tsconfig and Modules

What is it?

`tsconfig.json` is the settings file for the TypeScript checker. It says how strict to be, which features to allow, and how your files connect.

This chapter also covers TypeScript's extras for **modules** (JavaScript chapter 29): importing types, and **type definitions** for packages and for Node itself.

Why does it matter?

The same code can pass or fail depending on the settings. With `"strict": false`, TypeScript misses whole families of bugs. Understanding the settings means you can read any project's `tsconfig.json`, and you'll know why a line that's fine in one project is an error in another.

Real-world example

Think of the settings on a **spell-checker**:

Spell-checker settings	tsconfig
Language: British or American English	<code>target</code> and <code>lib</code> : which JavaScript features exist
"Also check grammar"	<code>"strict": true</code>
A dictionary for medical words	<code>@types/node</code> : extra words (types) for Node

How it works

Your playground settings, line by line

Here's the `tsconfig.json` from [chapter 01](#):

```
{
  "compilerOptions": {
    "target": "esnext",
    "module": "nodenext",
    "lib": ["esnext", "dom"],
    "strict": true,
    "noEmit": true,
    "allowImportingTsExtensions": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true
  }
}
```

Setting	What it means
<code>target: "esnext"</code>	You're writing modern JavaScript. Allow the newest features.
<code>module: "nodenext"</code>	Follow Node's module rules, including <code>"type": "module"</code> in <code>package.json</code> .
<code>lib: ["esnext", "dom"]</code>	Which built-in things exist: modern JavaScript, plus browser things like <code>document</code> and <code>fetch</code> .
<code>strict: true</code>	Turn on all the important safety checks (next section).
<code>noEmit: true</code>	Only check. Don't create <code>.js</code> files (Node runs <code>.ts</code> directly).
<code>allowImportingTsExtensions: true</code>	Allow <code>import ... from "./cart.ts"</code> , with <code>.ts</code> , which Node needs.
<code>erasableSyntaxOnly: true</code>	Warn about TypeScript features Node can't run, like <code>enum</code> (chapter 06).
<code>verbatimModuleSyntax: true</code>	Make you mark type-only imports clearly (see below).

What `strict` turns on

`"strict": true` is a bundle of checks. The ones you've already met:

Check	Catches
<code>strictNullChecks</code>	Using something that might be <code>null</code> or <code>undefined</code> (chapter 02)
<code>noImplicitAny</code>	Parameters with no type (chapter 05)
<code>strictPropertyInitialization</code>	Class properties that never get a value (chapter 07)

Always start new projects with `"strict": true`. Turning it on later, in a big project, means fixing hundreds of errors at once.

Extra checks worth knowing

These aren't part of `strict`, but many teams add them:

- `"noUncheckedIndexedAccess": true` makes `array[i]` return `T | undefined`, fixing the "trusting array indexes" mistake from [chapter 03](#). With it, `colors[1].toUpperCase()` gives `'c'` is possibly `'undefined'`.
- `"noUnusedLocals": true` reports variables you declared but never used.

Try adding one to your playground, run `npm tsc`, and see what it finds.

Importing and exporting

Modules work exactly like in JavaScript chapter 29. Types can be exported too:

```
// no-check
// cart.ts
export interface Product {
  name: string;
  price: number;
}

export function total(items: Product[]): number {
  return items.reduce((sum, product) => sum + product.price, 0);
}
```

One difference: with Node running `.ts` files directly, the import path uses the `.ts` extension.

import type

`Product` is a type, and types disappear when the code runs. So when you import one, you mark it with `type`:

```
// no-check
// main.ts
import type { Product } from "./cart.ts";
import { total } from "./cart.ts";

const items: Product[] = [{ name: "Mug", price: 12 }];
console.log(total(items)); // prints: 12
```

Forget the `type`, and `verbatimModuleSyntax` reminds you:

```
error TS1484: 'Product' is a type and must be imported using a type-only import when
'verbatimModuleSyntax' is enabled.
```

Why bother? When Node strips the types, it needs to know which imports to delete. `import type` tells it: "this whole line is only for the checker".

You can also mix both in one line: `import { total, type Product } from "./cart.ts";`.

Type definitions: `.d.ts` files and `@types`

Many npm packages are written in JavaScript. How does TypeScript know their types? From **type definition files**, which end in `.d.ts`. They contain only types, no code, like a menu without the kitchen.

- Many packages include their own `.d.ts` files. You install them, and the types just work.
- For others, the community writes definitions, published as `@types/<name>`, like `@types/lodash`.

Node itself is one of these. Using `process` fails at first:

```
// no-check
console.log(process.argv.length);
// ✗ Cannot find name 'process'. Do you need to install type definitions for node? ...
```

The fix is two steps:

1. Install the definitions: `npm install --save-dev @types/node`
2. Tell TypeScript to use them, by adding `"types": ["node"]` to `compilerOptions`.

(Recent TypeScript versions need step 2. Older tutorials only mention step 1.)

When you need `.js` files

Node runs `.ts` directly, but browsers don't, and npm packages are usually published as JavaScript. Then you *do* want `tsc` (or a tool like Vite, JavaScript chapter 50) to write `.js` files. In that case, you'd remove `noEmit`, and

set an `outDir`, the folder where the `.js` files go:

```
{
  "compilerOptions": {
    "outDir": "dist"
  }
}
```

For browser projects, Vite handles this for you ([chapter 12](#)).

Common mistakes

1. Turning `strict` off to make errors go away

The errors are bugs TypeScript found for you. Fix them, don't hide them.

2. Importing a type without `type`

With `verbatimModuleSyntax`, write `import type { Product }`, or `import { type Product }`.

3. Leaving out `.ts` in import paths

`import { total } from "./cart"` fails when Node runs it. Write `"./cart.ts"`.

4. Installing `@types/node` and nothing else

Also add `"types": ["node"]` to your `tsconfig.json`.

Quick recap

- `tsconfig.json` holds the checker's settings. Always use `"strict": true`.
- `strict` includes `strictNullChecks`, `noImplicitAny` and `strictPropertyInitialization`.
- Import types with `import type`, and use `.ts` in import paths when Node runs your files.
- `.d.ts` files and `@types/...` packages describe the types of JavaScript code.
- For Node's own types: install `@types/node` and add `"types": ["node"]`.

Next: try the [exercises](#), then move on to [12 TypeScript in the Browser](#).

Exercises

How to do these:

- Work in `playground/ch11/`. Some exercises have several files: make a subfolder for each (`ch11/ex1/` , ...).
 - Run with `node` , and check types with `npx tsc` from inside `playground` .
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Explain your settings

Without looking at the notes, write a comment next to each line of your playground's `tsconfig.json` explaining what it does in your own words. Then check your answers against the table in the notes.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Split into modules

Create two files in `ch11/ex2/` :

- `library.ts` exports `interface Book (title, author)` and a function `formatBook(book: Book): string`.
- `main.ts` imports both and prints two books.

Expected output of `node ch11/ex2/main.ts` :

```
"Dune" by Frank Herbert
"Coco" by Lee Unkrich
```

`npx tsc` must be quiet.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Stricter checks

Add `"noUncheckedIndexedAccess": true` to your `tsconfig.json` and run `npx tsc` .

1. Did any of your earlier exercises get new errors? For each one, write down why TypeScript thinks the value might be `undefined` .
2. Fix them properly: check for `undefined` instead of using `!` or `as` .

(If nothing breaks, write this code and fix it:)

```
// no-check
const temperatures = [21, 24, 19];
console.log(temperatures[0].toFixed(1));
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): A typed command-line tool

Make `ch11/ex4/greet.ts`, which reads a name from the command line:

```
node ch11/ex4/greet.ts Asha
```

prints:

```
Hello, Asha!
```

and with no name, prints `Hello, stranger!`.

1. Write it using `process.argv` (JavaScript chapter 49), and read the error from `npx tsc`.
2. Fix the error with the two steps from the notes.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

12 TypeScript in the Browser

What is it?

TypeScript knows every part of the browser's **DOM** (JavaScript chapter 20): every element type, every event, every property. This chapter shows how to work with those types, and how to run TypeScript in a web page.

Why does it matter?

DOM code is full of hidden "what if"s. What if `querySelector` doesn't find anything? What if the element isn't an input, so it has no `.value`? In JavaScript, you find out when the page crashes. In TypeScript, the checker asks you these questions up front.

Real-world example

A **parcel locker** at a shop:

Parcel locker	DOM types
You ask for locker 12	<code>document.querySelector("#locker-12")</code>
It might be empty	The result might be <code>null</code>
The screen says "Parcel", but is it a letter or a box?	<code>Element</code> , but is it an <code>HTMLInputElement</code> ?
Check the label before you open it	A <code>null</code> check, then <code>instanceof</code>

How it works

Elements might be `null`

```
const button = document.querySelector("#buy");
button.addEventListener("click", () => {});
// ❌ 'button' is possibly 'null'.
```

`querySelector` returns `null` when nothing matches: a typo in the selector, or a script that runs before the HTML exists. TypeScript makes you handle that:

```
const button = document.querySelector("#buy");

if (button === null) {
  throw new Error("Missing #buy button"); // fail loudly, as in JavaScript chapter 18
}

button.addEventListener("click", () => {
  console.log("Added to cart");
});
```

After the check, TypeScript knows `button` is an element.

Which kind of element?

Different elements have different properties. Inputs have `.value`, images have `.src`. But an ID alone doesn't tell TypeScript what kind of element it is:

```
const input = document.getElementById("email");
if (input !== null) {
  console.log(input.value);
  // ✗ Property 'value' does not exist on type 'HTMLElement'.
}
```

There are three ways to tell TypeScript the element's type:

1. Select by tag name. TypeScript knows what `"input"` means:

```
const input = document.querySelector("input"); // HTMLInputElement | null
console.log(input?.value);
```

2. Check with `instanceof`. This really checks, at run time, so it's the safest:

```
const input = document.getElementById("email");
if (input instanceof HTMLInputElement) {
  console.log(input.value); // fine: input is an HTMLInputElement here
}
```

3. Say it with a generic. Quick, but it's a promise TypeScript can't check:

```
const input = document.querySelector<HTMLInputElement>("#email"); // HTMLInputElement | null
console.log(input?.value);
```

Common element types:

Element	Type	Useful properties
<code><input></code>	<code>HTMLInputElement</code>	<code>value</code> , <code>checked</code>
<code><button></code>	<code>HTMLButtonElement</code>	<code>disabled</code>
<code></code>	<code>HTMLImageElement</code>	<code>src</code> , <code>alt</code>
<code><form></code>	<code>HTMLFormElement</code>	<code>reset()</code>
<code><select></code>	<code>HTMLSelectElement</code>	<code>value</code>
Anything else	<code>HTMLElement</code>	<code>textContent</code> , <code>classList</code> , <code>style</code>

Events

Event listeners get typed events for free:

```
document.addEventListener("keydown", (event) => {
  // event: KeyboardEvent, so TypeScript knows about event.key
  if (event.key === "Escape") {
    console.log("Closing the menu");
  }
});
```

You didn't write `KeyboardEvent`. TypeScript knows that `"keydown"` events are keyboard events, the same contextual typing as in [chapter 05](#).

`event.target` needs care, though. It could be *anything* on the page:

```
const form = document.querySelector("form");
form?.addEventListener("input", (event) => {
  console.log(event.target.value);
  // ✗ 'event.target' is possibly 'null'.
  // ✗ Property 'value' does not exist on type 'EventTarget'.
});
```

Two problems: the target might be `null`, and even if it isn't, TypeScript only knows it's an `EventTarget`, which could be a button, a div, or the whole page.

Narrow it with `instanceof`, like before:

```
const form = document.querySelector("form");
form?.addEventListener("input", (event) => {
  if (event.target instanceof HTMLInputElement) {
    console.log(`${event.target.name} is now ${event.target.value}`);
  }
});
```

This fits nicely with event delegation from JavaScript chapter 21: one listener, and an `instanceof` check to see what was actually clicked.

Running TypeScript in a web page

Browsers can't run `.ts` files. Something has to turn them into JavaScript first. The easiest tool is **Vite** (JavaScript chapter 50):

```
npm create vite@latest
```

Choose **Vanilla**, then **TypeScript**. Then:

```
cd my-project
npm install
npm run dev
```

Your code lives in `src/main.ts`. Every time you save, Vite converts it and refreshes the page. VS Code shows type errors as you type, and `npx tsc` checks the whole project.

(Vite's prompts can change a little between versions, but the idea stays the same.)

Common mistakes

1. Using `!` to silence `null` errors

```
const button = document.querySelector("#buy")!; // "trust me, it's there"
```

The `!` (called the **non-null assertion**) tells TypeScript "this isn't null". If you're wrong, the page still crashes. Prefer a real check that throws a clear error.

2. Using `as` when `instanceof` would check for real

`document.getElementById("email") as HTMLInputElement` is a promise, not a check. If the element is actually a `<div>`, TypeScript believes you, and the bug waits for run time. `instanceof` checks for real.

3. Using `event.target` without narrowing

`event.target` can be any element, or even `null`. Check it with `instanceof` before using element-specific properties.

Quick recap

- `querySelector` and `getElementById` can return `null`. Check before you use the result.
- Different elements have different types, like `HTMLInputElement` and `HTMLImageElement`.
- Tag selectors, `instanceof` checks, and `querySelector<Type>` all tell TypeScript the element type. `instanceof` is the safest.
- Event listeners get typed events automatically, but `event.target` must be narrowed.
- Browsers can't run `.ts` files. Use Vite with its TypeScript template.

Next: try the [exercises](#), then move on to [13 Typing Async Code and APIs](#).

Exercises

How to do these:

- Create one Vite project for this chapter: `npm create vite@latest` inside `playground`, name it `ch12`, and choose **Vanilla** and **TypeScript**. Then `cd ch12`, `npm install`, `npm run dev`.
- For each exercise, replace the `<body>` of `ch12/index.html` with the HTML given, and write your code in `src/main.ts`. (Delete the starter code Vite put there.)
- `npx tsc` (inside `ch12`) must show no errors. **Rule:** no `!` and no `as`. Use real checks.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): A like button

```
<body>
  <button id="like">👍 0</button>
  <script type="module" src="/src/main.ts"></script>
</body>
```

Each click adds one like and updates the button's text: 👍 1, 👍 2, ...

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Character counter

```
<body>
  <textarea id="bio" maxlength="100"></textarea>
  <p id="count">0 / 100</p>
  <script type="module" src="/src/main.ts"></script>
</body>
```

As the user types, update the paragraph: 42 / 100 .

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A typed tip calculator

```
<body>
  <form id="tip-form">
    <input name="bill" type="number" placeholder="Bill" required>
    <select name="percent">
      <option value="10">10%</option>
      <option value="15" selected>15%</option>
      <option value="20">20%</option>
    </select>
    <button>Calculate</button>
  </form>
  <p id="result"></p>
  <script type="module" src="/src/main.ts"></script>
</body>
```

On submit, show Tip: \$7.50 for a bill of 50 at 15%. Write the calculation as a separate typed function, `calculateTip(bill: number, percent: number): number`, that doesn't touch the page.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Challenge): Delegated delete buttons

```
<body>
  <ul id="tasks">
    <li data-id="1">Buy milk <button class="delete">X</button></li>
    <li data-id="2">Call mum <button class="delete">X</button></li>
    <li data-id="3">Water plants <button class="delete">X</button></li>
  </ul>
  <p id="status"></p>
  <script type="module" src="/src/main.ts"></script>
</body>
```

Use **one** click listener on the `<ul>` (event delegation). When a delete button is clicked, remove its `<li>` and show `Deleted task 2` (with the right id) in the status paragraph. Clicking the text of a task should do nothing.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

13 Typing Async Code and APIs

What is it?

This chapter adds types to code that waits (JavaScript chapters 31–33): promises, `async` functions, and data from APIs with `fetch`.

The big idea: data from the outside world has **no guaranteed type**. TypeScript can only protect you if you check it when it arrives.

Why does it matter?

`response.json()` gives back whatever the server sent. TypeScript can't see the server, so by default it calls the result `any`, and all checking silently stops. One renamed field on the server, and your app shows `undefined` everywhere, with no warning.

Typing API data properly is one of the most valuable things TypeScript does in real apps.

Real-world example

A **parcel from an online shop**:

Parcel	API data
The order confirmation says "1 blue mug"	Your <code>interface Mug</code>
The box that actually arrives	The JSON from the server
Assuming the box matches the order	<code>const data: Mug = await response.json()</code>
Opening it and checking before you pay	A type guard that checks the data

How it works

Async functions return promises

An `async` function always returns a promise. Its return type is `Promise<...>`, with the value type in the brackets (a generic, [chapter 08](#)):

```
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function loadScore(): Promise<number> {
  await wait(100); // pretend this is a slow server
  return 42;
}

const score = await loadScore(); // score: number
console.log(score); // prints: 42
```

(Top-level `await` works because our files are modules.) Forget the `Promise`, and TypeScript tells you:

```
async function loadScore(): number {
  // ❌ The return type of an async function or method must be the global Promise<T> type. Did
  // you mean to write 'Promise<number>'?
  return 42;
}
```

The `any` hiding in `fetch`

```
// no-check
const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
const user = await response.json(); // user: any
console.log(user.nmae); // a typo, and no error: prints undefined
```

Hover over `user`: it's `any`. The typo `nmae` goes straight through.

Option 1: say what you expect

You can annotate the result:

```
interface User {
  id: number;
  name: string;
  email: string;
}

const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
const user: User = await response.json();
console.log(user.name); // prints: Leanne Graham
```

Now `user.nmae` would be an error. But be honest about what this is: a **promise to TypeScript**, not a check. If the server sends something different, TypeScript still believes you. It's fine for APIs you trust and control.

Option 2: treat it as `unknown`, then check

For data you don't control, start from `unknown` ([chapter 02](#)) and check its shape with a **type guard**: a function that returns `value is User`:

```
interface User {
  id: number;
  name: string;
  email: string;
}

function isUser(value: unknown): value is User {
  return (
    typeof value === "object" &&
    value !== null &&
    "id" in value && typeof value.id === "number" &&
    "name" in value && typeof value.name === "string" &&
    "email" in value && typeof value.email === "string"
  );
}

const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
const data: unknown = await response.json();

if (isUser(data)) {
  console.log(`${data.name} <${data.email}>`); // prints: Leanne Graham <Sincere@april.biz>
} else {
  console.log("The server sent something unexpected");
}
```

`value is User` means: "if this function returns `true`, treat `value` as a `User`". Inside the `if`, TypeScript narrows `data` to `User`, like the narrowing in [chapter 06](#), but with your own check.

Writing guards by hand gets long for big objects. In real projects, people use validation libraries like **Zod**, which build the check and the type from one description. Now you know what they do inside.

A reusable, typed `fetchJson`

Put the pieces together in one generic helper. It checks `response.ok` (remember, `fetch` doesn't reject on 404, JavaScript chapter 33), and takes a guard for the data:

```

interface Todo {
  id: number;
  title: string;
  completed: boolean;
}

function isTodo(value: unknown): value is Todo {
  return (
    typeof value === "object" && value !== null &&
    "id" in value && typeof value.id === "number" &&
    "title" in value && typeof value.title === "string" &&
    "completed" in value && typeof value.completed === "boolean"
  );
}

async function fetchJson<T>(url: string, guard: (value: unknown) => value is T): Promise<T> {
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }
  const data: unknown = await response.json();
  if (!guard(data)) {
    throw new Error("Unexpected data from the server");
  }
  return data;
}

const todo = await fetchJson("https://jsonplaceholder.typicode.com/todos/1", isTodo); // todo:
  Todo
console.log(todo.title, todo.completed); // prints: delectus aut autem false

```

Errors in `catch` are `unknown`

In strict mode, the `error` in a `catch` block has the type `unknown`, because JavaScript lets you throw *anything*, not only errors:

```

try {
  JSON.parse("{ not json}");
} catch (error) {
  console.log(error.message);
  // ❌ 'error' is of type 'unknown'.
}

```

Narrow it with `instanceof Error`:

```
try {
  JSON.parse("{ not json}");
} catch (error) {
  if (error instanceof Error) {
    console.log("Could not read the data:", error.name); // prints: Could not read the data:
    SyntaxError
  }
}
```

Results that can succeed or fail

A clean pattern for "it worked, or it didn't" is a discriminated union ([chapter 06](#)):

```
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string };

function parsePrice(text: string): Result<number> {
  const price = Number(text);
  if (Number.isNaN(price)) {
    return { ok: false, error: `${text} is not a number` };
  }
  return { ok: true, data: price };
}

const result = parsePrice("abc");
if (result.ok) {
  console.log(result.data * 2);
} else {
  console.log(result.error); // prints: "abc" is not a number
}
```

TypeScript won't let you read `result.data` until you've checked `result.ok`. You can't forget the failure case.

Common mistakes

1. Leaving API data as `any`

If you never type `response.json()`, you've switched TypeScript off for the most important data in your app. Annotate it, or better, check it.

2. Thinking an annotation checks the data

`const user: User = await response.json()` is a promise to TypeScript. Only a real check (a type guard, or a library like Zod) catches bad data at run time.

3. Using `error.message` without narrowing

`catch (error)` gives you `unknown`. Check `error instanceof Error` first.

4. Writing `: number` on an async function

Async functions return `Promise<number>`, not `number`.

Quick recap

- Async functions return `Promise<T>`. Write it as the return type.
 - `response.json()` is `any` by default, so type it, or TypeScript stops checking.
 - An annotation is a promise. A type guard (`value is User`) is a real check.
 - A generic `fetchJson<T>` with a guard gives you checked, typed data everywhere.
 - `catch (error)` gives `unknown`: narrow with `instanceof Error`.
 - `Result<T>` unions make you handle both success and failure.
-

Next: try the [exercises](#), then move on to [14 Project: Typed Shopping Cart](#).

Exercises

How to do these:

- Work in `playground/ch13/`, one file per exercise. You need to be online.
 - Run with `node`, and check types with `npx tsc`. Every exercise must pass `npx tsc` with no errors.
 - **Rule:** no `any`, and no `as`.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Type the waiting

Add types to this code so `npx tsc` is quiet. Every function needs a return type.

```
// no-check
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function brewCoffee(size) {
  await wait(500);
  return `Your ${size} coffee is ready`;
}

console.log(await brewCoffee("large"));
```

Expected output (after about half a second):

```
Your large coffee is ready
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): A typed post

Fetch `https://jsonplaceholder.typicode.com/posts/1` and print its title. Create `interface Post` with `id`, `userId`, `title` and `body`, and annotate the result.

Expected output:

```
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
```

Then misspell `title` as `tilte` in your `console.log`, and check that `npx tsc` catches it.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Guard against bad data

Write `isPost(value: unknown): value is Post`, and a function `getPost(id: number): Promise<Post | null>` that returns `null` if the request fails (`!response.ok`) or the data isn't a post.

```
// no-check
const good = await getPost(1);
const missing = await getPost(9999);
console.log(good?.id, missing);
```

Expected output:

```
1 null
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): Results instead of exceptions

Write `fetchResult<T>(url: string, guard: (value: unknown) => value is T): Promise<Result<T>>`, using the `Result<T>` type from the notes. It never throws: a network error, a bad status, or bad data all become `{ ok: false, error: "..."}` .

Then fetch user 1's todos from `https://jsonplaceholder.typicode.com/todos?userId=1`. You'll need a guard for an **array** of todos.

Expected output:

```
User 1 has 20 todos, 11 done
```

Also try a broken URL like `https://jsonplaceholder.typicode.com/nope`, and print the error message instead.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

14 Project: Typed Shopping Cart

What you'll build

You'll take the shopping cart you built in JavaScript chapter 12 and convert it to TypeScript, one step at a time. The program's output stays **exactly the same**. What changes is that TypeScript now guards every part of it.

This is what developers do in real life: most TypeScript projects started as JavaScript projects. Converting step by step, without breaking anything, is a skill you'll use a lot.

By the end, you'll have:

- typed data (`Product` , `CartItem` , `DiscountCode`),
- typed functions, where wrong calls are caught before running,
- a discount code that can only ever be `"SAVE10"` or `"FLAT5"` ,
- the code split into modules,
- tests for the money logic.

Before you start

- Finish (or at least reread) your JavaScript chapter 12 `cart.js` . If you don't have it any more, do that project first: it's the starting point.
- Make a folder `playground/ch14/` and copy your `cart.js` into it.
- **The golden rule:** after every milestone, run both `node` (the output must not change) and `npx tsc` (the errors should go down).

Save a copy of your program's full output now, in a file like `expected-output.txt` , so you can compare after each step.

Milestone 1: Rename and look

Goal: see what TypeScript thinks of your JavaScript.

1. Rename `cart.js` to `cart.ts` .
2. Run `node ch14/cart.ts` . It should still work: remember, Node just removes types, and there aren't any yet.
3. Run `npx tsc` , and count the errors.

Most of them will be `Parameter 'productId' implicitly has an 'any' type.` , plus a few about empty arrays and `null` . That's your to-do list.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 2: Type the data

Goal: describe your data with types ([chapter 04](#)).

Create these at the top of `cart.ts` :

```
// no-check
type Cents = number; // all money is whole cents, as in JavaScript chapter 12

interface Product {
  id: number;
  name: string;
  priceInCents: Cents;
}

interface CartItem {
  productId: number;
  quantity: number;
}
```

Then use them: `const products: Product[] = [...]` and `const cart: CartItem[] = []` ; .

`type Cents = number` doesn't add any checking (a `Cents` is still any number), but it documents what the number means, everywhere it appears.

Check: the errors about the empty `cart` array should be gone.

Milestone 3: Type the helper functions

Goal: give every function typed parameters and a return type ([chapter 05](#)).

Start with the helpers, since the other functions use them:

- `findProduct(productId: number): Product | null`
- `findCartItem(productId: number): CartItem | null`
- `formatMoney(cents: Cents): string`

Now TypeScript will point at every place that uses `findProduct` without checking for `null` . Each of those is a real crash waiting to happen. Fix them with a `null` check ([chapter 06](#)).

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 4: A discount code that can't be misspelled

Goal: replace "any string" with exactly the valid codes ([chapter 06](#)).

```
// no-check
type DiscountCode = "SAVE10" | "FLAT5";

let discountCode: DiscountCode | null = null;
```

Now `getDiscount(subtotal: Cents, code: DiscountCode | null): Cents` can't be called with `"SAVE10"` by mistake.

But there's a catch: `applyDiscountCode` receives whatever the customer typed, which is a plain `string`, like `"save10"`. After cleaning it up, how do you get from `string` to `DiscountCode`? Write a type guard ([chapter 13](#)):

```
// no-check
function isDiscountCode(value: string): value is DiscountCode {
  return value === "SAVE10" || value === "FLAT5";
}
```

Use it inside `applyDiscountCode`: if the cleaned code passes, store it. If not, print your "Sorry, that code isn't valid" message, as before.

Check: `node` output unchanged, and fewer `tsc` errors.

Milestone 5: Type the totals

Goal: type the bigger functions.

Your `getTotals()` returns an object. Give its shape a name:

```
// no-check
interface Totals {
  subtotal: Cents;
  discount: Cents;
  tax: Cents;
  shipping: Cents;
  total: Cents;
}
```

Then type the rest: `getSubtotal(): Cents`, `getTax(amount: Cents): Cents`, `getShipping(subtotal: Cents): Cents`, `getTotals(): Totals`, and the `void` functions like `printReceipt()`, `addToCart`, `removeFromCart` and `updateQuantity`.

Check: `npx tsc` should now show **zero** errors, and the output is still exactly the same. Compare it with your `expected-output.txt`.

Milestone 6: Split into modules

Goal: give each file one job ([chapter 11](#)).

```
ch14/
├─ types.ts      Product, CartItem, DiscountCode, Totals, Cents
├─ products.ts   the catalog
├─ cart.ts       the cart and all the logic
└─ main.ts       the calls that run the program, and print the receipt
```

Remember `import type` for types, and `.ts` in import paths. Run `node ch14/main.ts`: same output as always.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 7: Test the money

Goal: protect the logic with tests (JavaScript chapter 46).

Create `ch14/cart.test.ts` and test `getDiscount`, `getTax` and `getShipping`. For example:

```
// no-check
import { test } from "node:test";
import assert from "node:assert/strict";
import { getDiscount } from "./cart.ts";

test("SAVE10 takes 10% off, rounded to the nearest cent", () => {
  assert.equal(getDiscount(5647, "SAVE10"), 565);
});

test("FLAT5 needs a subtotal over $30", () => {
  assert.equal(getDiscount(2500, "FLAT5"), 0);
});
```

Run `node --test` inside `ch14`. Test files import `node:test`, so TypeScript needs Node's types: install `@types/node` and add `"types": ["node"]` to your `tsconfig.json` (chapter 11).

Try this too: write `getDiscount(5647, "SAVE20")` in a test. You don't even need to run it: TypeScript rejects it straight away.

Common mistakes

1. Changing behavior while converting

A conversion should only add types. If the output changes, undo the last step and try again. Keep your `expected-output.txt` handy.

2. Using `any` or `as` to make errors disappear

Every error in Milestone 1 is a question. Answer it with a real type or a real check.

3. Converting everything at once

Work in small steps, running `node` and `npx tsc` after each one. When something breaks, you'll know exactly which step did it.

Quick recap

- Real projects often move from JavaScript to TypeScript step by step: rename, type the data, type the functions, then tighten.
- Type the data first, since everything else depends on it.
- `null` errors from TypeScript point at real crashes: fix them with checks.
- Literal unions plus a type guard turn "any string" into "only valid values".
- Modules and tests work the same as in JavaScript, with `import type` and `.ts` paths.

Congratulations! You've finished the TypeScript folder. 🎉 Try the [exercises](#) for extra features, then go back to the roadmap. React with TypeScript is a great next step.

Exercises

How to do these:

- These are extra features for your typed cart. Finish all 7 milestones in the notes first.
- After each exercise, `npx tsc` must show no errors, and `node --test` must still pass.
- **Rule:** no `any`, and no `as`.
- When you're done, ask Claude to review your code.

Exercise 1 (Easy): A read-only catalog

Nothing in the program should ever change the product catalog. Make TypeScript enforce that: mark the array and each product's properties read-only.

Then try `products[0].priceInCents = 1;` in `main.ts`, read the error, and remove it.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Codes from one list

Right now, the valid codes are written twice: once in `type DiscountCode`, and once in `isDiscountCode`. Write them **once**, as an array with `as const`, and build both the type and the guard from it.

The program's output must not change.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Results instead of messages

`addToCart` currently prints its error messages itself. That makes it hard to test. Change it to return a `Result` (chapter 13), and let `main.ts` do the printing:

```
// no-check
type AddResult =
  | { ok: true; item: CartItem }
  | { ok: false; reason: "unknown-product" | "bad-quantity" };
```

Then write tests for all three outcomes.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): A new discount type

Add a third discount code, `BUY2`: the cheapest item in the cart is free when there are at least 2 items (counting quantities).

1. Add it to your codes list from Exercise 2.

2. Run `npx tsc`, and let TypeScript show you every place that needs updating. (If you used a `switch` with a `never` check in `getDiscount`, it will point right at it.)
3. Add tests for `BUY2`: with 1 item (no discount), and with several items.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Hints

Try each exercise on your own first. Come here only when you're stuck, and read one hint at a time.

01 Getting Started

Exercise 1 (Easy): Set up and say hello

[Back to the exercise](#)

Hint

Any JavaScript file is already valid TypeScript. `console.log` works exactly like before.

Exercise 2 (Easy): Label the boxes

[Back to the exercise](#)

Hint

The three basic types are `string`, `number` and `boolean`. The annotation goes after the name: `const seat: number = 14;`.

Exercise 3 (Medium): Make TypeScript complain

[Back to the exercise](#)

Hint

Reread the section "Types disappear when the code runs".

Exercise 4 (Challenge): Catch the chapter 38 bug

[Back to the exercise](#)

Hint 1

Keep the raw text in its own variable, like `const quantityText: string = "3";`, then make a second variable `const quantity: number = ... using Number()`.

Hint 2

Try writing `const quantity: number = "3";` first, and read the error. That's the protection you get.

02 Basic Types

Exercise 1 (Easy): Infer or annotate?

[Back to the exercise](#)

Hint

Hover over each variable in VS Code. If it shows `any`, TypeScript couldn't infer it, and that one needs a type.

Exercise 2 (Easy): Fix the red lines

[Back to the exercise](#)

Hint

Numbers don't have quotes. Booleans are only `true` or `false`.

Exercise 3 (Medium): A value that can be missing

[Back to the exercise](#)

Hint

The type is `string | null`. An `if (borrowedBy === null)` check works exactly like in JavaScript chapter 07.

Exercise 4 (Challenge): `unknown` input

[Back to the exercise](#)

Hint

Use `typeof input === "string"` and `typeof input === "number"` checks. Inside each `if`, TypeScript knows the real type.

03 Arrays, Tuples and Objects

Exercise 1 (Easy): A typed shopping list

[Back to the exercise](#)

Hint

An empty array needs its type written out: `const list: string[] = []`; `.join(", ")` is from JavaScript chapter 10.

Exercise 2 (Easy): Map pins

[Back to the exercise](#)

Hint

The type of one pin is `[string, number, number]`. The type of the whole list is `[string, number, number][]`. In the loop: `for (const [name, lat, lng] of pins)`.

Exercise 3 (Medium): A typed movie list

[Back to the exercise](#)

Hint 1

The type of the array is `{ title: string; year: number; rating?: number }[]`.

Hint 2

`rating` might be `undefined`, so check it before printing, or use `??` from JavaScript chapter 07.

Exercise 4 (Challenge): Bank accounts with read-only IDs

[Back to the exercise](#)

Hint

`readonly` goes in front of the property name in the type: `{ readonly id: number; owner: string; balance: number }[]`. The balance is not read-only, so you can update it.

04 Type Aliases and Interfaces

Exercise 1 (Easy): Name that shape

[Back to the exercise](#)

Hint

Once you have `interface Song { ... }`, the array type becomes `Song[]`.

Exercise 2 (Easy): Type aliases for clarity

[Back to the exercise](#)

Hint

`type Coordinates = [number, number];`. Use destructuring to get the two numbers out of `to`.

Exercise 3 (Medium): Extending

[Back to the exercise](#)

Hint

`interface Trainer extends Member { clients: string[]; }.`

Exercise 4 (Challenge): The same thing, two ways

[Back to the exercise](#)

Hint

`type Trainer = Member & { clients: string[] };`. For question 2, look at the comparison table in the notes.

05 Functions

Exercise 1 (Easy): Add the types

[Back to the exercise](#)

Hint

Two functions, five parameters. Each parameter gets `: number` or `: string`, and each function gets a return type after its `)`.

Exercise 2 (Easy): Optional and default

[Back to the exercise](#)

Hint

The optional one goes last: `snack?: string`. Inside, check `if (snack)` before adding `,` with `...`.

Exercise 3 (Medium): A function type for discounts

[Back to the exercise](#)

Hint

`type Discount = (price: number) => number;` For "never below 0", `Math.max(0, ...)` from JavaScript chapter 05 is handy.

Exercise 4 (Challenge): A typed grade report

[Back to the exercise](#)

Hint 1

`Math.round(x * 100) / 100` rounds to 2 decimals and keeps the result a number, so there are no extra zeros.

Hint 2

Guard against dividing by zero first: `if (scores.length === 0) return 0;`

06 Unions and Narrowing

Exercise 1 (Easy): Price tags

[Back to the exercise](#)

Hint

Narrow with `typeof value === "string"`, and convert with `Number()` in that branch. The array's type is `(number | string)[]`.

Exercise 2 (Easy): Traffic light

[Back to the exercise](#)

Hint

A `switch (light)` with three `case` s works nicely here.

Exercise 3 (Medium): Order status messages

[Back to the exercise](#)

Hint

Three interfaces, each with a literal `status`, like `status: "shipped"`. Then `type Order = Pending | Shipped | Delivered;` and a `switch (order.status)`.

Exercise 4 (Challenge): Never forget a case

[Back to the exercise](#)

Hint

The `default` branch is `const unhandled: never = order; return unhandled;`. The error in step 2 should mention your new interface by name.

07 Classes

Exercise 1 (Easy): Type the class

[Back to the exercise](#)

Hint

Three declarations above the constructor (`title: string;` and so on), three parameter types, and a return type for `summary`.

Exercise 2 (Easy): A piggy bank

[Back to the exercise](#)

Hint

`private coins = 0;` declares and starts it in one line. Try both `private coins` and `#coins`, and notice which one still hides the value when you `console.log(bank)`.

Exercise 3 (Medium): Payment methods with `implements`

[Back to the exercise](#)

Hint

```
class CardPayment implements PaymentMethod { last4: string; constructor(last4: string) { ... }  
pay(amount: number): string { ... } }.
```

Exercise 4 (Challenge): An abstract employee

[Back to the exercise](#)

Hint 1

A child class's constructor must call `super(name)` before using `this`.

Hint 2

Declare the child's own properties (like `hourlyRate: number;`) inside the child class, not in `Employee`.

08 Generics

Exercise 1 (Easy): last

[Back to the exercise](#)

Hint

`.at(-1)` from JavaScript chapter 10 already returns `undefined` for an empty array.

Exercise 2 (Easy): A typed box

[Back to the exercise](#)

Hint

`const books: Box<string[]> = { label: "Books", contents: ["Dune", "Coco"] }; . Then books.contents.length works, because TypeScript knows it's an array.`

Exercise 3 (Medium): Group by a key

[Back to the exercise](#)

Hint

Create `new Map<string, number>()`, loop over the items, and use `(counts.get(key) ?? 0) + 1`.

Exercise 4 (Challenge): A tiny typed store

[Back to the exercise](#)

Hint 1

Inside the class: `private items: T[] = [];`. The class's `<T>` works just like a function's.

Hint 2

`remove` can replace the list with a filtered copy: `this.items = this.items.filter(...)`.

09 Utility Types

Exercise 1 (Easy): Edit a recipe

[Back to the exercise](#)

Hint

The `changes` parameter is `Partial<Recipe>`.

Exercise 2 (Easy): Hide the secret

[Back to the exercise](#)

Hint 1

```
Omit<Account, "passwordHash">.
```

Hint 2

TypeScript's types don't remove anything at run time. You still have to build the new object yourself, for example with `const { passwordHash, ...rest } = account;` from JavaScript chapter 15.

Exercise 3 (Medium): Opening hours

[Back to the exercise](#)

Hint

The value type is `string | null`: `Record<Day, string | null>`.

Exercise 4 (Challenge): Settings with defaults

[Back to the exercise](#)

Hint

Keep the defaults in a `const defaults: Settings = { ... }`, then return `{ ...defaults, ...saved }`.

10 keyof, typeof and Mapped Types

Exercise 1 (Easy): Menu keys

[Back to the exercise](#)

Hint

`type Dish = keyof typeof menu;` Read it right to left: the type of `menu`, then its keys.

Exercise 2 (Easy): A list and a union from one line

[Back to the exercise](#)

Hint

`type Level = (typeof levels)[number];` For `isLevel`, `levels.includes(value as Level)` works, because `includes` on a `readonly Level[]` expects a `Level`.

Exercise 3 (Medium): A type-safe sort

[Back to the exercise](#)

Hint

Use `toSorted` (JavaScript chapter 13) with a compare function that compares `a[key]` and `b[key]` with `<` and `>`, returning `-1`, `1` or `0`.

Exercise 4 (Challenge): A form's error messages

[Back to the exercise](#)

Hint

`type FormErrors<T> = { [K in keyof T]?: string };` Start with `const errors: FormErrors<SignUpForm> = {};` and add properties with `if` checks.

11 tsconfig and Modules

Exercise 1 (Easy): Explain your settings

[Back to the exercise](#)

Hint

JSON doesn't allow comments, but `tsconfig.json` is special: TypeScript accepts `//` comments in it.

Exercise 2 (Easy): Split into modules

[Back to the exercise](#)

Hint

Two import lines: `import type { Book } from "../library.ts";` and `import { formatBook } from "../library.ts";`.

Exercise 3 (Medium): Stricter checks

[Back to the exercise](#)

Hint

An `if (value !== undefined)` check, or `?.`, or `?? 0` for a default. Each one narrows away `undefined`.

Exercise 4 (Challenge): A typed command-line tool

[Back to the exercise](#)

Hint 1

`process.argv[2]` is the first word after the file name. It might be `undefined`, so use `??`.

Hint 2

`npm install --save-dev @types/node`, then `"types": ["node"]` inside `compilerOptions`.

12 TypeScript in the Browser

Exercise 1 (Easy): A like button

[Back to the exercise](#)

Hint

`document.querySelector("button")` gives an `HTMLButtonElement | null`. Throw a clear error if it's `null`, then add the click listener.

Exercise 2 (Easy): Character counter

[Back to the exercise](#)

Hint

The textarea's type is `HTMLTextAreaElement`. Use `instanceof HTMLTextAreaElement` to narrow the result of `getElementById("bio")`, and listen for the `input` event.

Exercise 3 (Medium): A typed tip calculator

[Back to the exercise](#)

Hint 1

Input and select values are always strings (JavaScript chapter 22). Convert them with `Number()` before calling `calculateTip`.

Hint 2

`form.elements.namedItem("bill")` returns something general. Narrow it with `instanceof HTMLInputElement`, and the other one with `instanceof HTMLSelectElement`.

Exercise 4 (Challenge): Delegated delete buttons

[Back to the exercise](#)

Hint 1

Start with `if (!(event.target instanceof HTMLButtonElement)) return;`, a guard clause.

Hint 2

`event.target.closest("li")` returns `HTMLLIElement | null`, and `li.dataset.id` is `string | undefined`.
Handle both.

13 Typing Async Code and APIs

Exercise 1 (Easy): Type the waiting

[Back to the exercise](#)

Hint

`wait` returns `Promise<void>`, and `brewCoffee` returns `Promise<string>`.

Exercise 2 (Easy): A typed post

[Back to the exercise](#)

Hint

```
const post: Post = await response.json();
```

Exercise 3 (Medium): Guard against bad data

[Back to the exercise](#)

Hint

Post 9999 doesn't exist: the server answers with status 404 and an empty object `{}`. Your `response.ok` check handles the status, and your guard would catch the `{}` too.

Exercise 4 (Challenge): Results instead of exceptions

[Back to the exercise](#)

Hint 1

An array guard: `Array.isArray(value) && value.every(isTodo)`, with `value is Todo[]` as its return type.

Hint 2

Wrap the whole fetch in `try / catch`, and return `{ ok: false, error: ... }` from the `catch`, using the `instanceof Error` check from the notes.

14 Project: Typed Shopping Cart

Milestone 1: Rename and look

[Back to the lesson](#)

Hint

Don't try to fix everything at once. The next milestones go through the errors in a sensible order: data first, then functions.

Milestone 3: Type the helper functions

[Back to the lesson](#)

Hint

If your JavaScript version used `find()`, it returns `undefined`, not `null`, when nothing matches. Either change the return type to `Product | undefined`, or add `?? null`. Pick one style and use it everywhere.

Milestone 6: Split into modules

[Back to the lesson](#)

Hint

Only export what other files need. `cart` itself can stay private inside `cart.ts`, if `main.ts` only uses the functions.

Exercise 1 (Easy): A read-only catalog

[Back to the exercise](#)

Hint

`readonly Product[]` stops changes to the list. `Readonly<Product>` (chapter 09) stops changes to each product. Combine them: `readonly Readonly<Product>[]`.

Exercise 2 (Easy): Codes from one list

[Back to the exercise](#)

Hint

Chapter 10: `const DISCOUNT_CODES = ["SAVE10", "FLAT5"] as const;` and `type DiscountCode = (typeof DISCOUNT_CODES)[number];` .

Exercise 3 (Medium): Results instead of messages

[Back to the exercise](#)

Hint

In `main.ts`, a `switch` on `result.reason` with a `never` check (chapter 06) makes sure every failure has a message.

Exercise 4 (Challenge): A new discount type

[Back to the exercise](#)

Hint 1

`getDiscount` will need the cart items, not only the subtotal. Change its parameters, and let the type errors guide you to every call that needs updating.

Hint 2

To find the cheapest item's price, map the cart items to their product prices first, then use `Math.min(...prices)` .